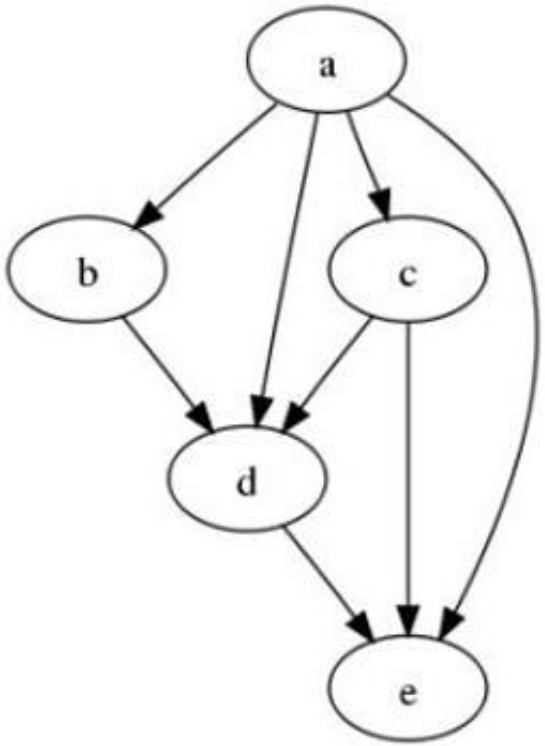
Orden Topológico, SCC, 2-SAT

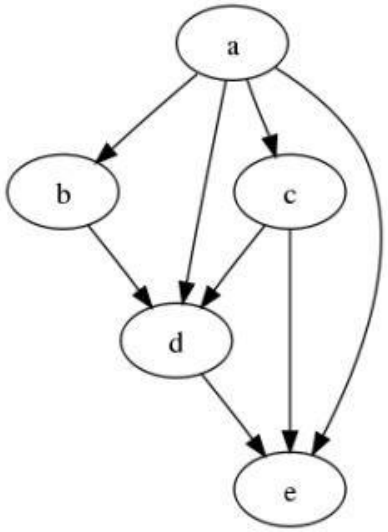
Oliver Winsor Pozo Flores

DAG(Directed Acyclic Graph)



Orden Topológico

Dado un grafo dirigido, un orden topológico es un ordenamiento de los nodos de modo que para cada arista x->y, x viene antes que y en el orden.



Este grafo tiene dos órdenes topológicos:

- a,b,c,d,e
- a,c,b,d,e

Orden Topológico

Un grafo tiene orden topológico si y sólo si no tiene ciclo. A estos grafos se los llama DAG (directed-acyclic-graph o “grafo dirigido sin ciclos”).

Para obtener el orden topológico en un DAG:

- Obtenemos el grado de entrada de cada nodo y metemos los de grado 0 en una cola.

- Mientras que la cola no esté vacía, tomamos el nodo del frente y “lo eliminamos” del grafo, actualizando el grado de entrada de sus vecinos y metiendo a la cola los que pasen a tener grado 0.

Orden Topológico

```
vector<int> topSort(vector<vector<int>> &g, vector<int> in)
{
    int n = in.size();
    vector<int> topo;
    queue<int> q;
    for(int i = 0; i < n; i++)
        if(in[i] == 0)
            q.push(i);
    while(!q.empty()) {
        int node = q.front(); q.pop();
        topo.push_back(node);
        for(int x : g[node])
            if(--in[x] == 0)
                q.push(x);
    }
    return topo;
}
```

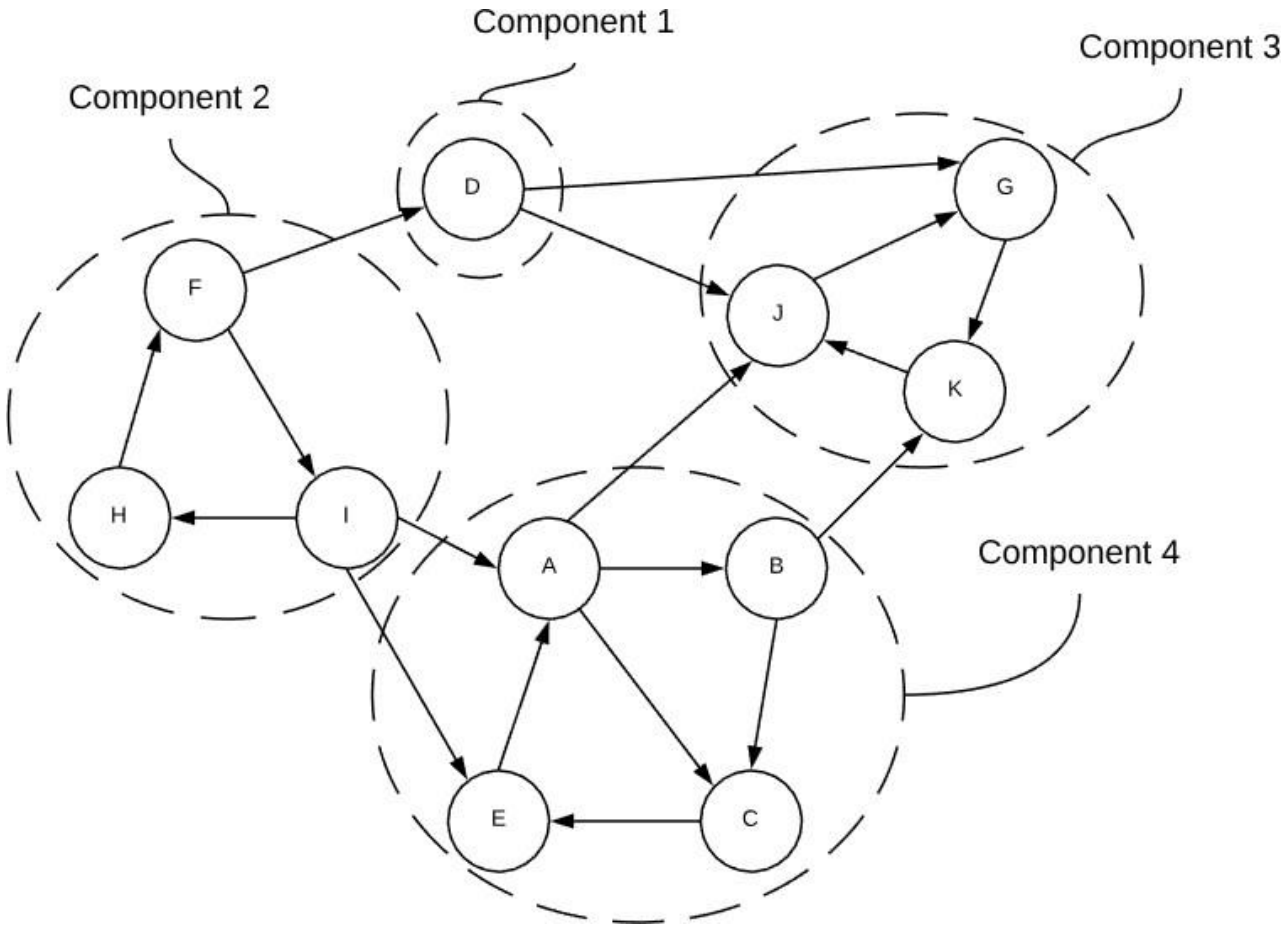
Problema Ejemplo

- https://codeforces.com/contest/1931/problem/F

Componentes fuertemente Conexas(SCC)

- Grafo fuertemente conexo: Un grafo dirigido G es llamado fuertemente conexo si para cada par de vértices u y v existe un camino de u hacia v y un camino de v hacia u.
- Componentes fuertemente conexas (Strongly Connected Components, SCC): Los componentes fuertemente conexos de un grafo dirigido son sus subgrafos maximales fuertemente conexos.

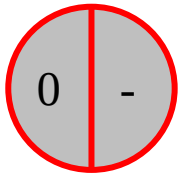
SCC


Component 1
Component 2
Component 3
Component 4
D
G
F
J
K
H
I
A
B
E
C

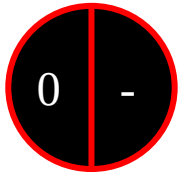
Kosaraju

- Con un primer DFS, obtenemos la lista de nodos en orden inverso de finalización (sería el orden topológico si fuera un DAG).
- Damos la vuelta todas las aristas de G (obteniendo G’). Esto no cambia las componentes fuertemente conexas, e invierte todas las aristas de la condensación de G.
- Continuamos con DFS partiendo de los nodos en el orden inverso, los demás nodos alcanzables por este formarán parte de la nueva componente fuertemente conexa.

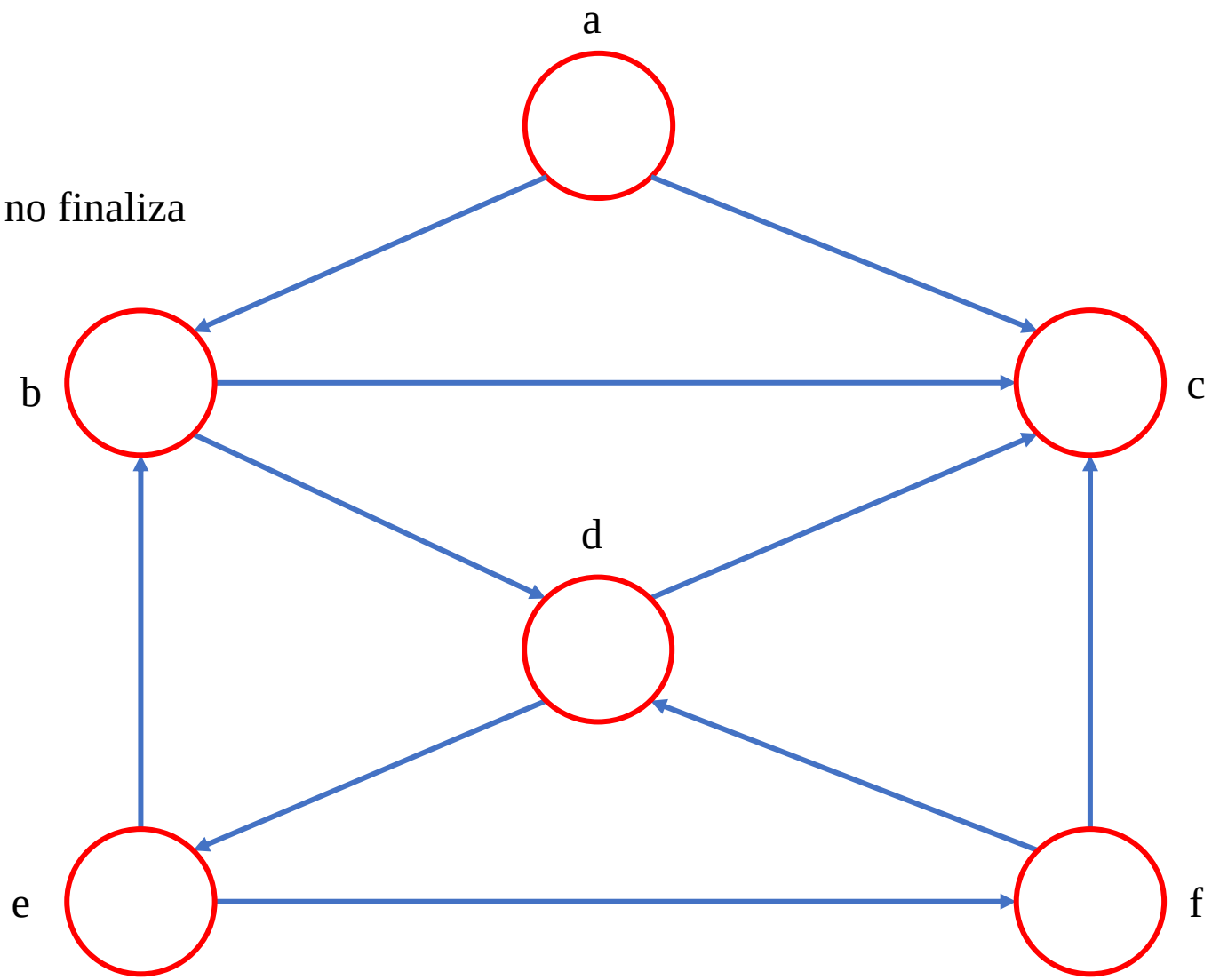
Kosaraju



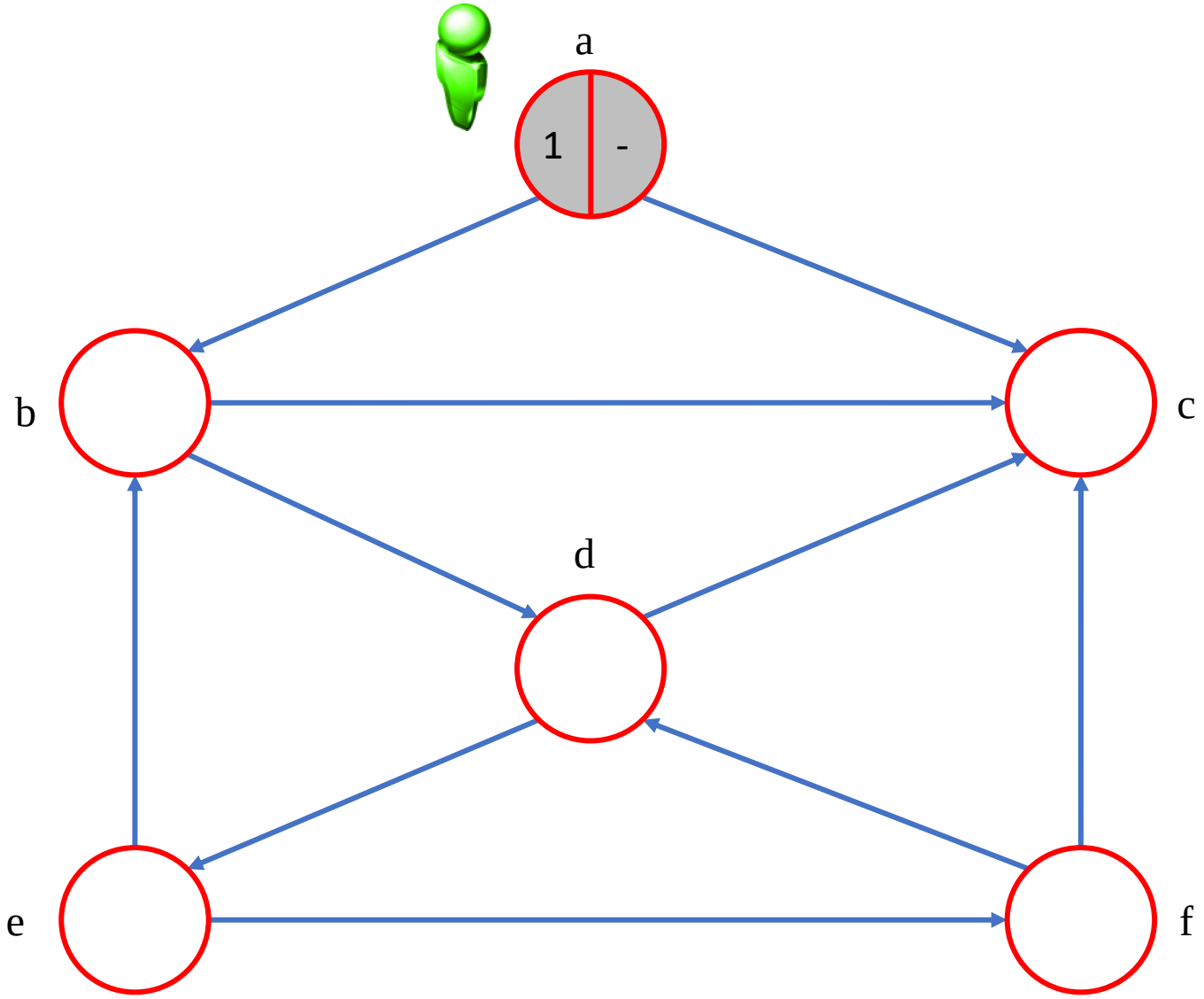
Vistos, todavía no finaliza



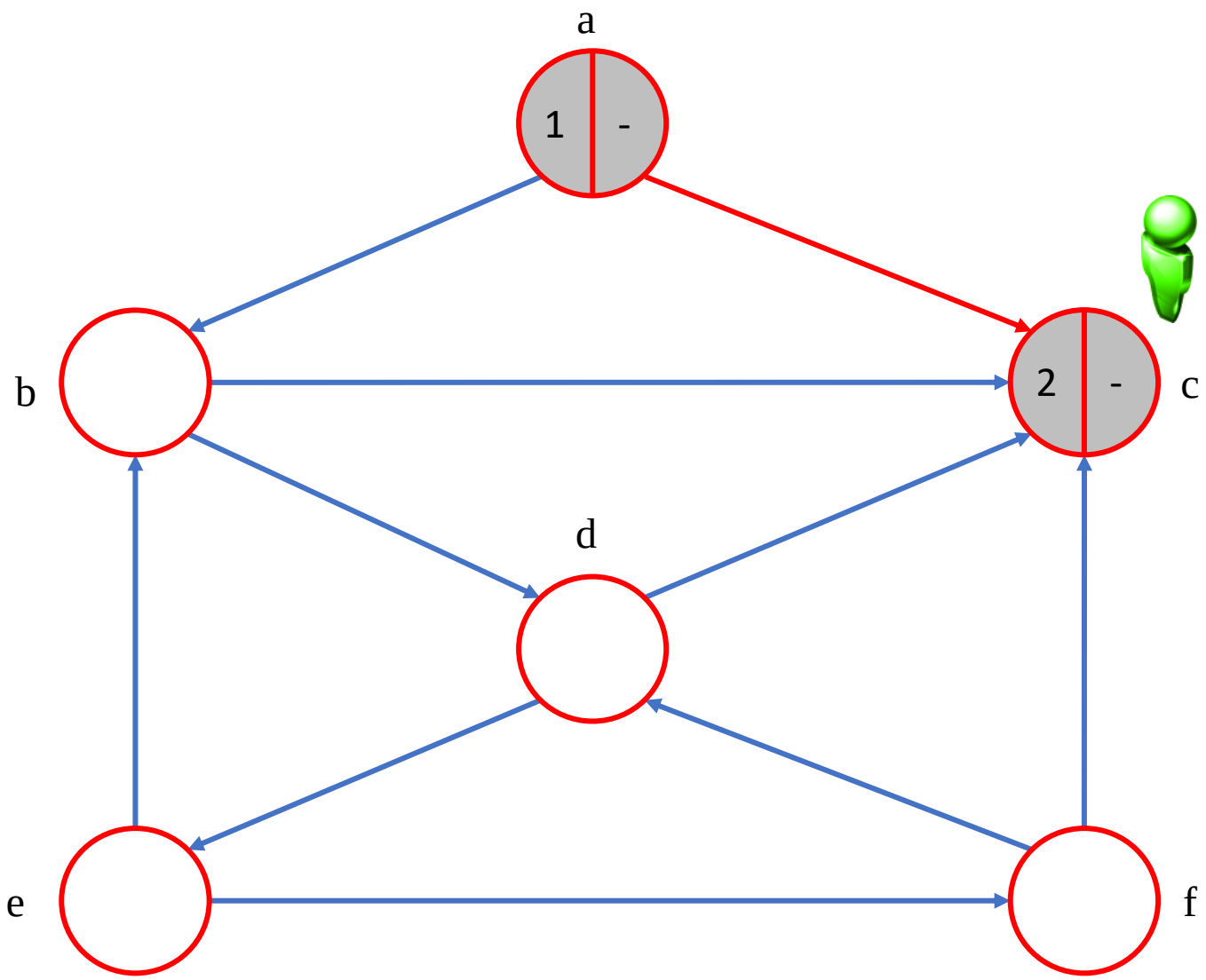
Finaliza



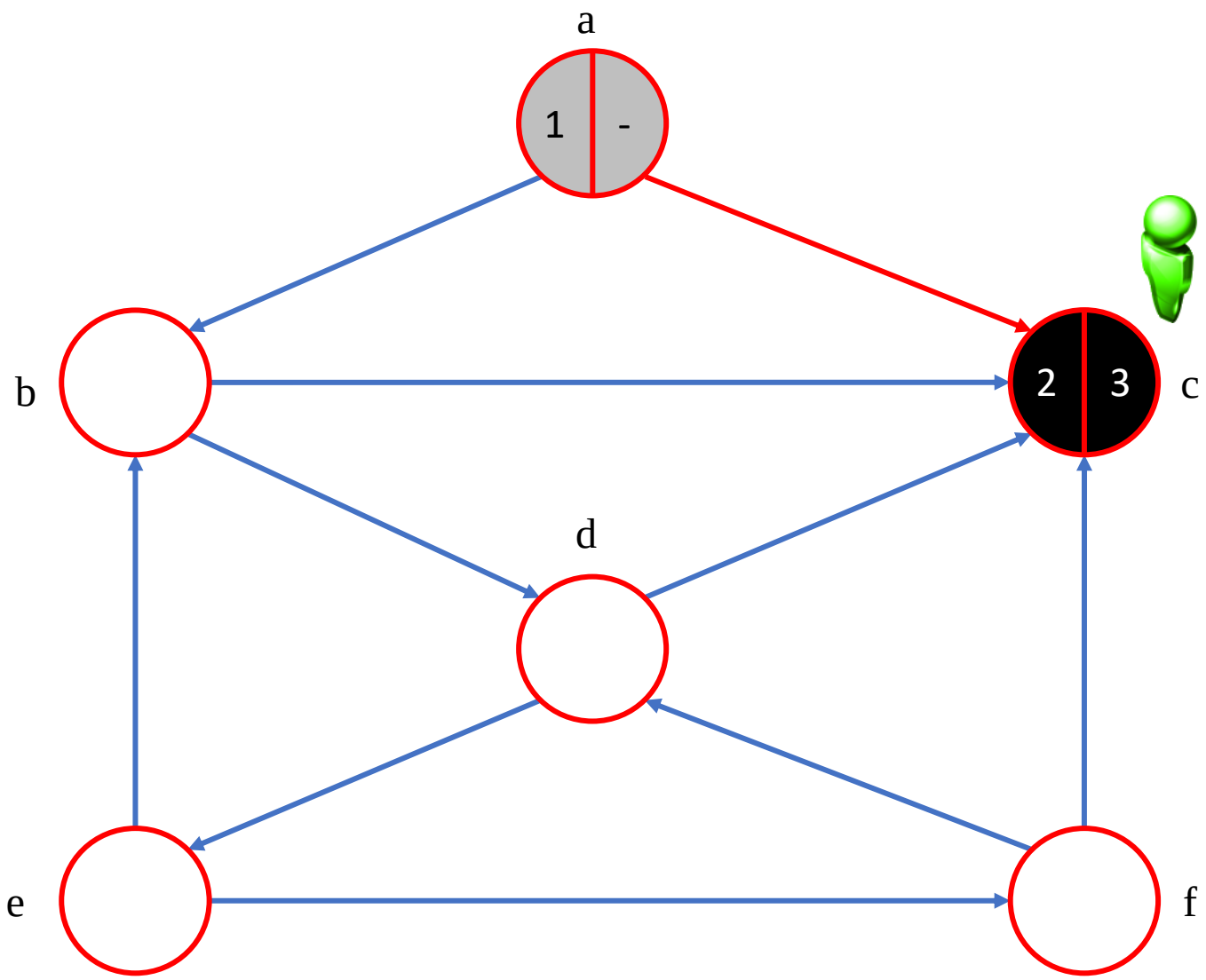
Kosaraju



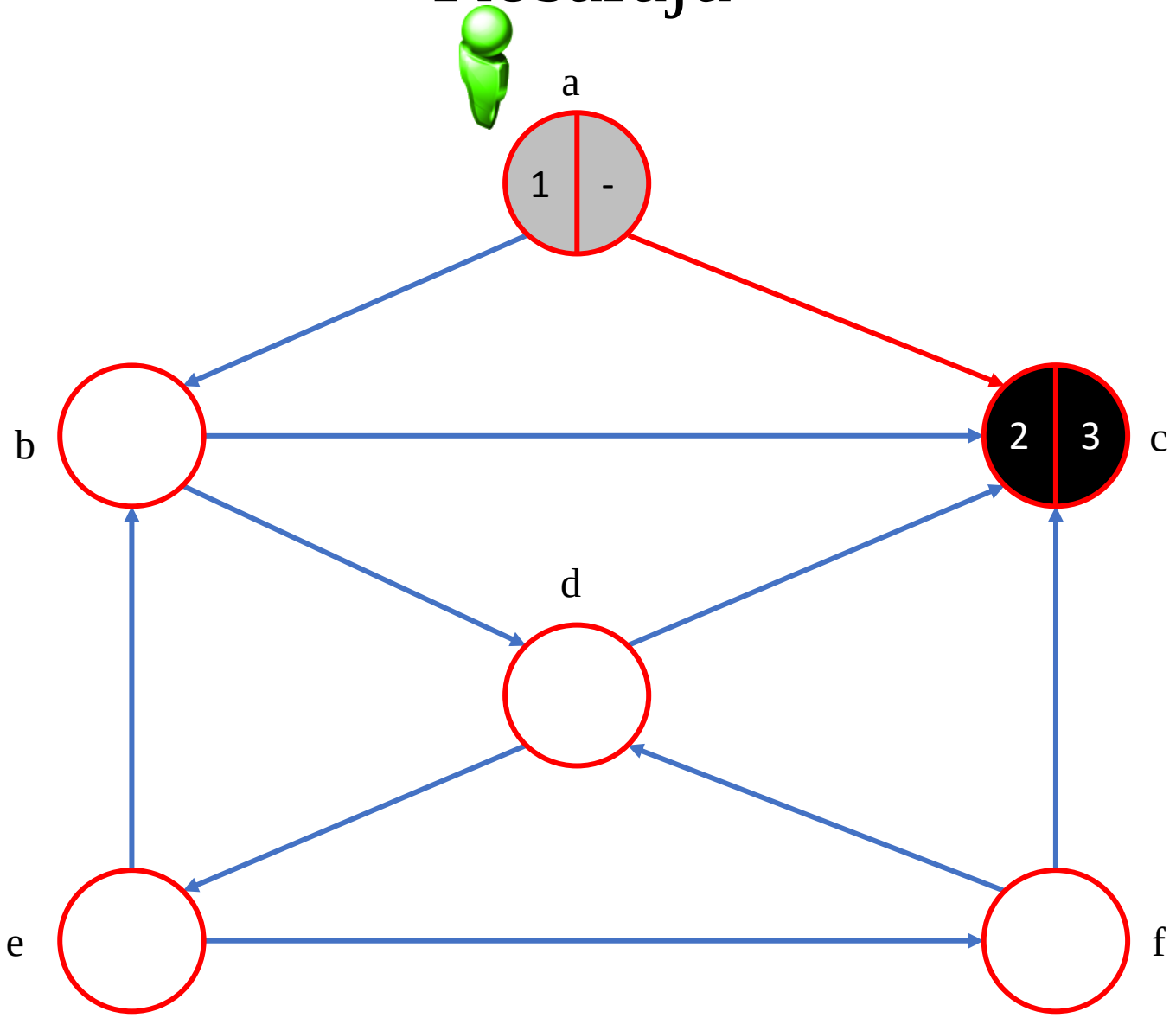
Kosaraju



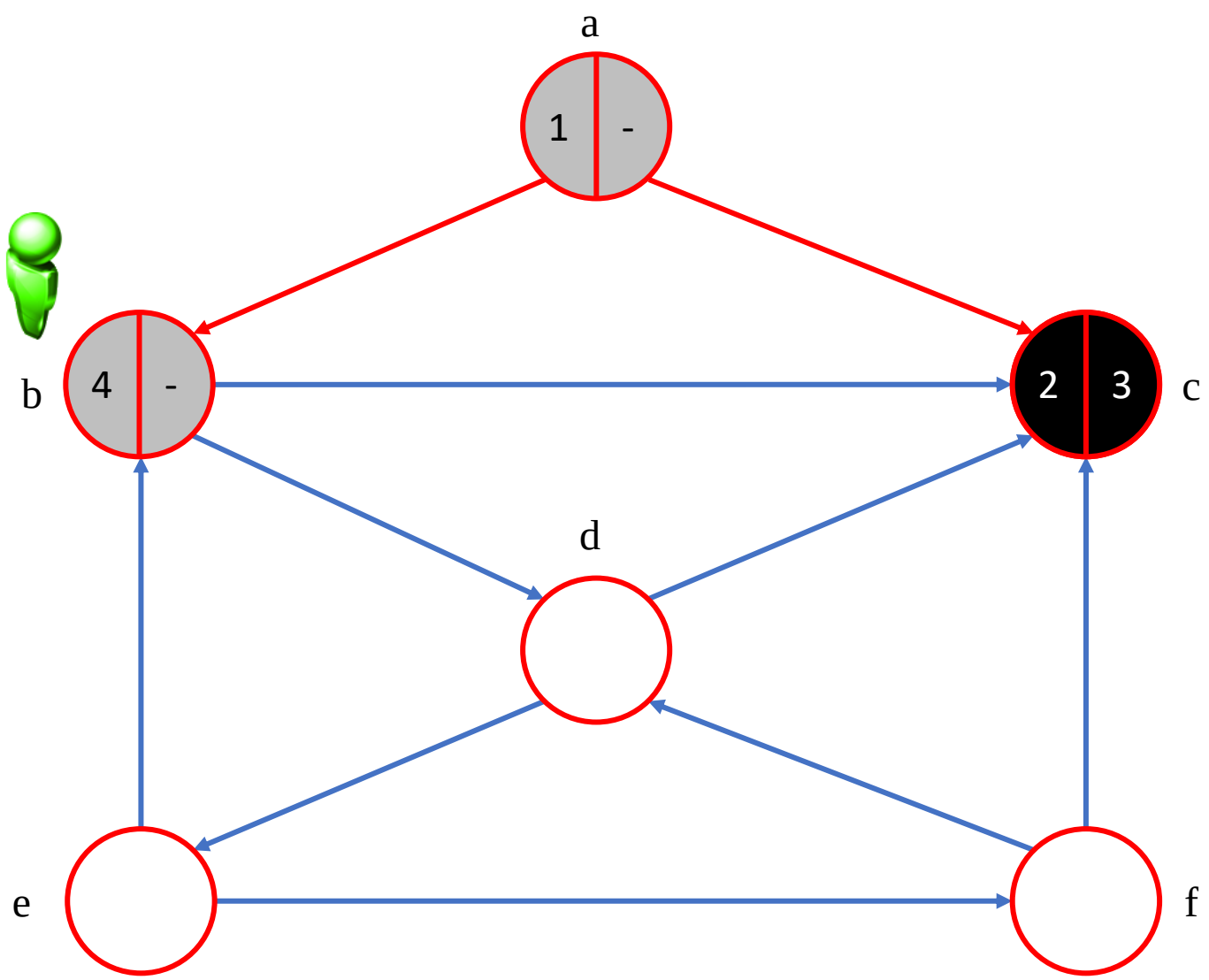
Kosaraju



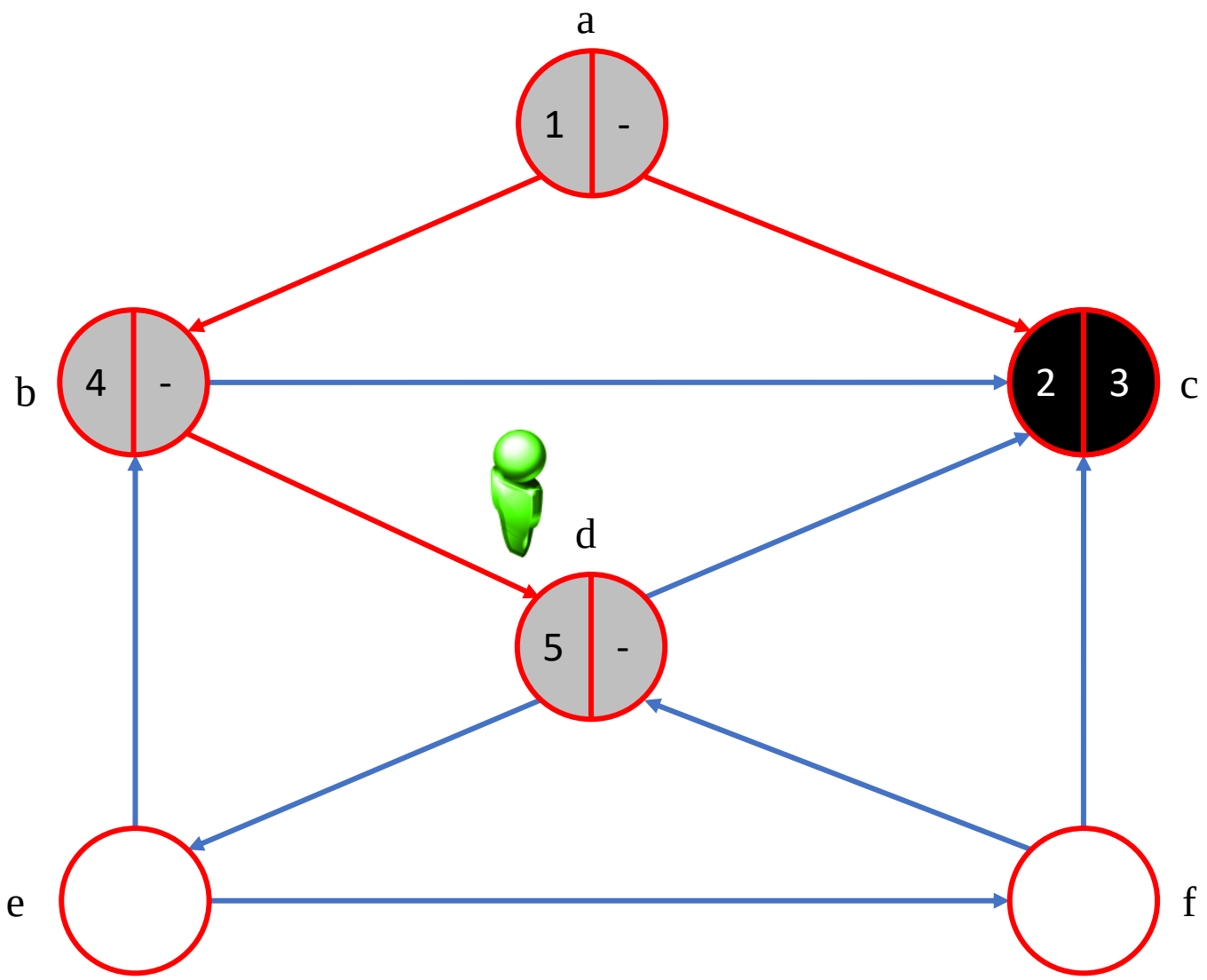
Kosaraju



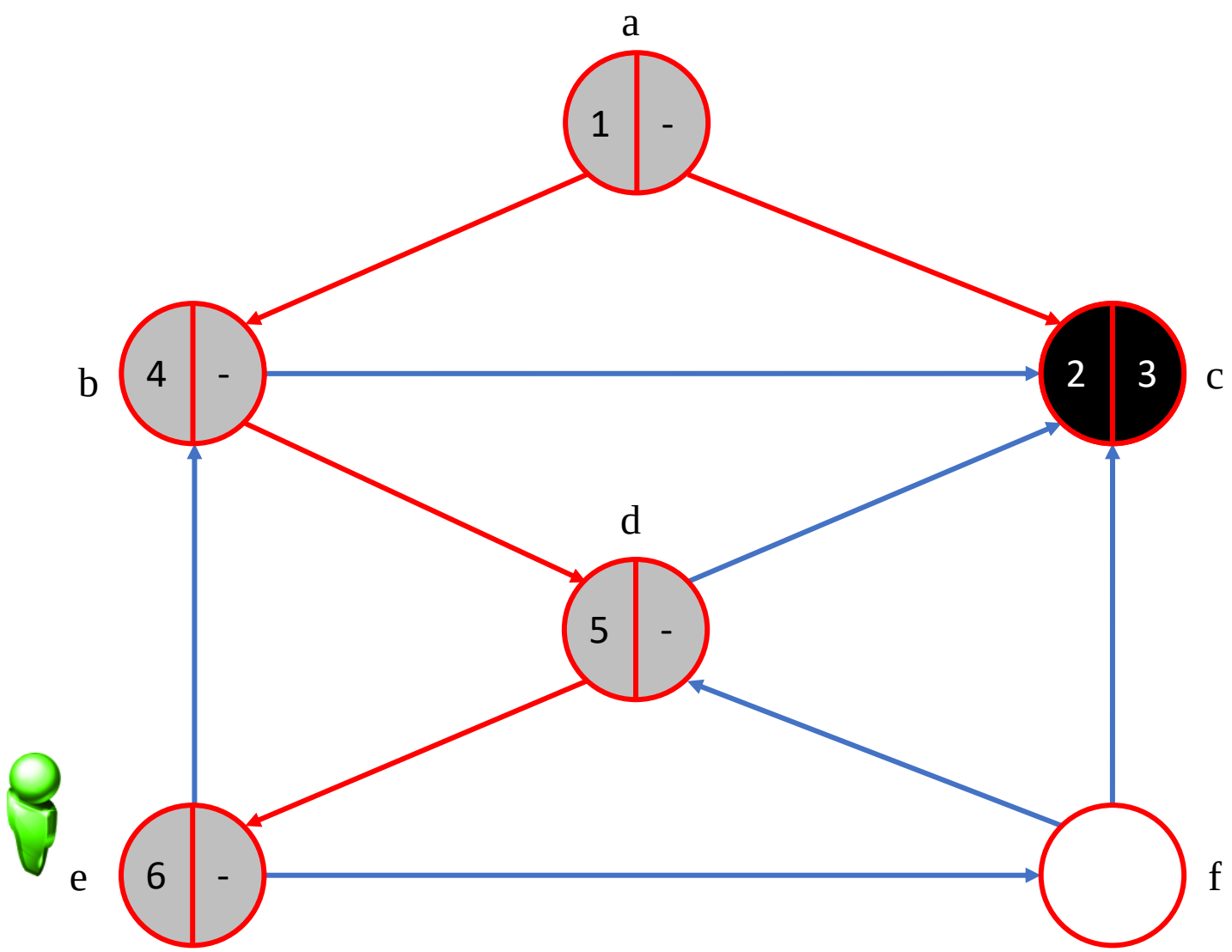
Kosaraju



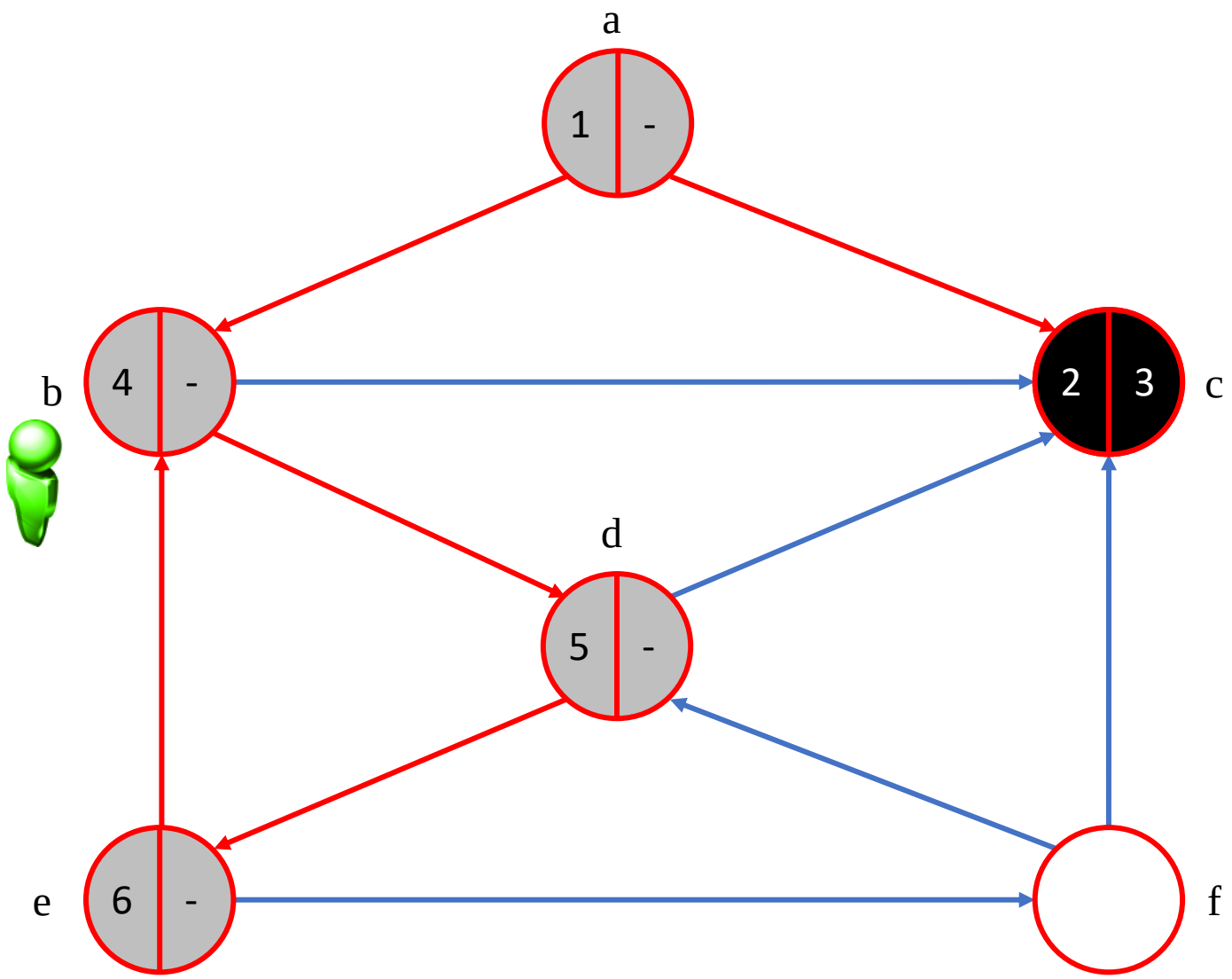
Kosaraju



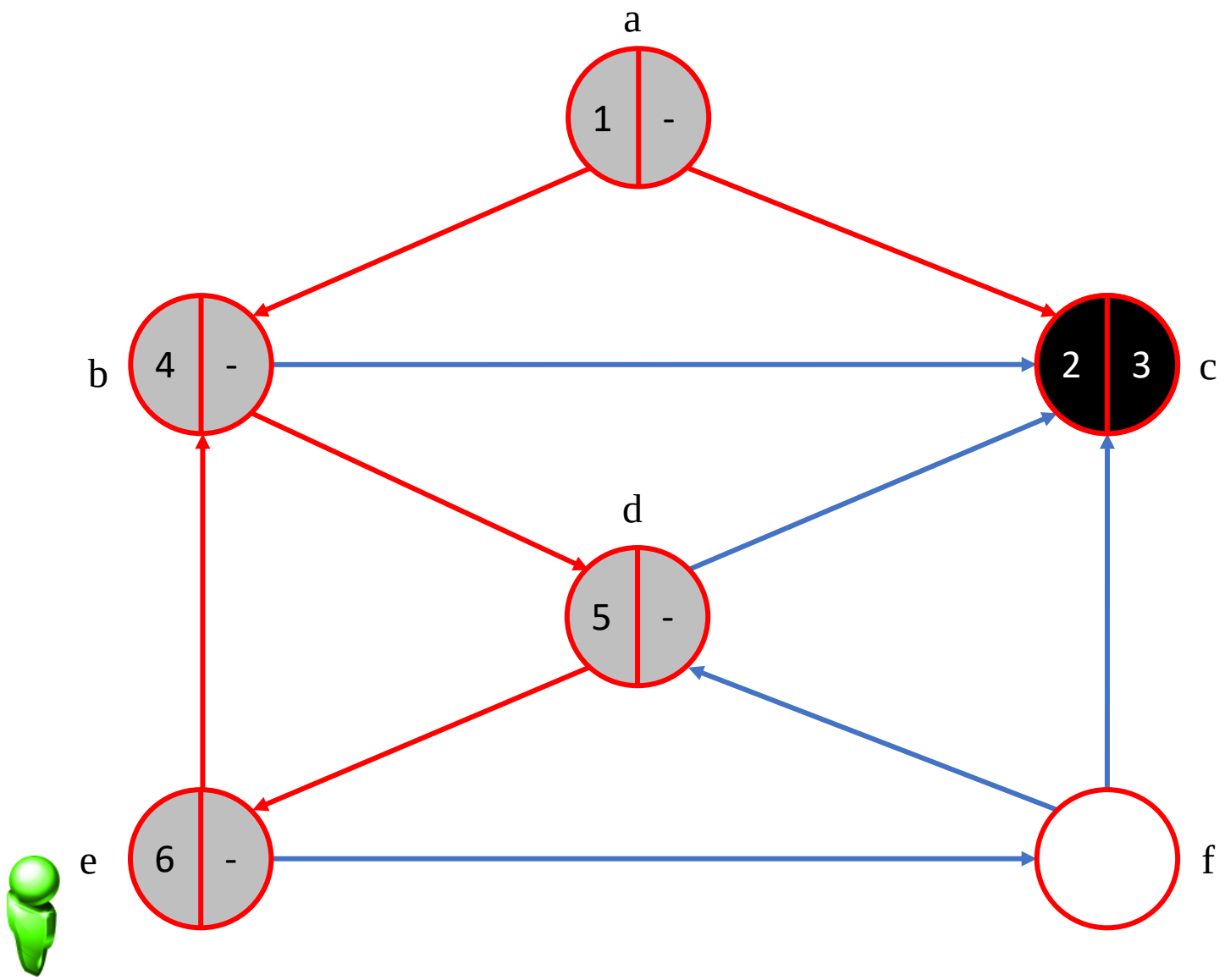
Kosaraju



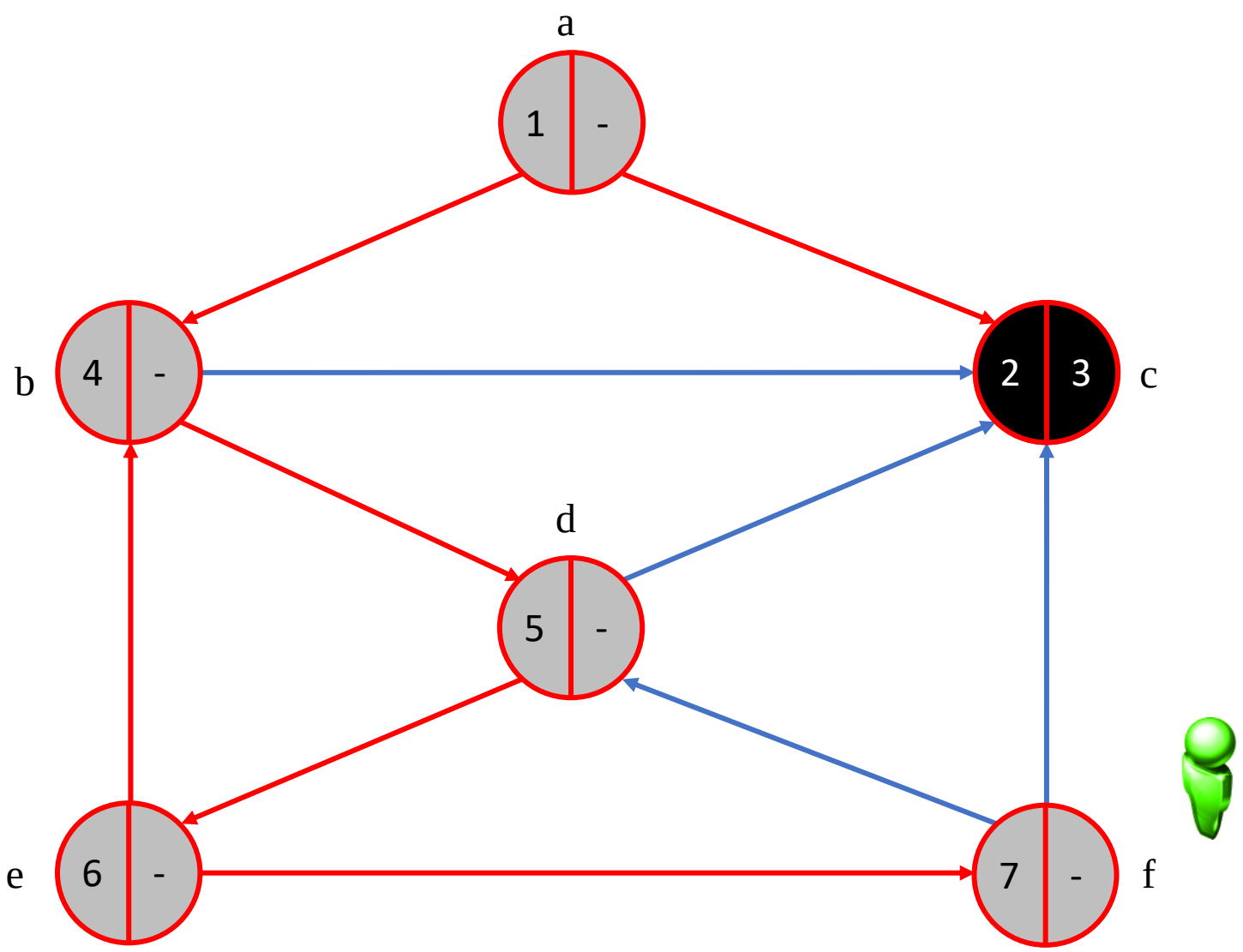
Kosaraju



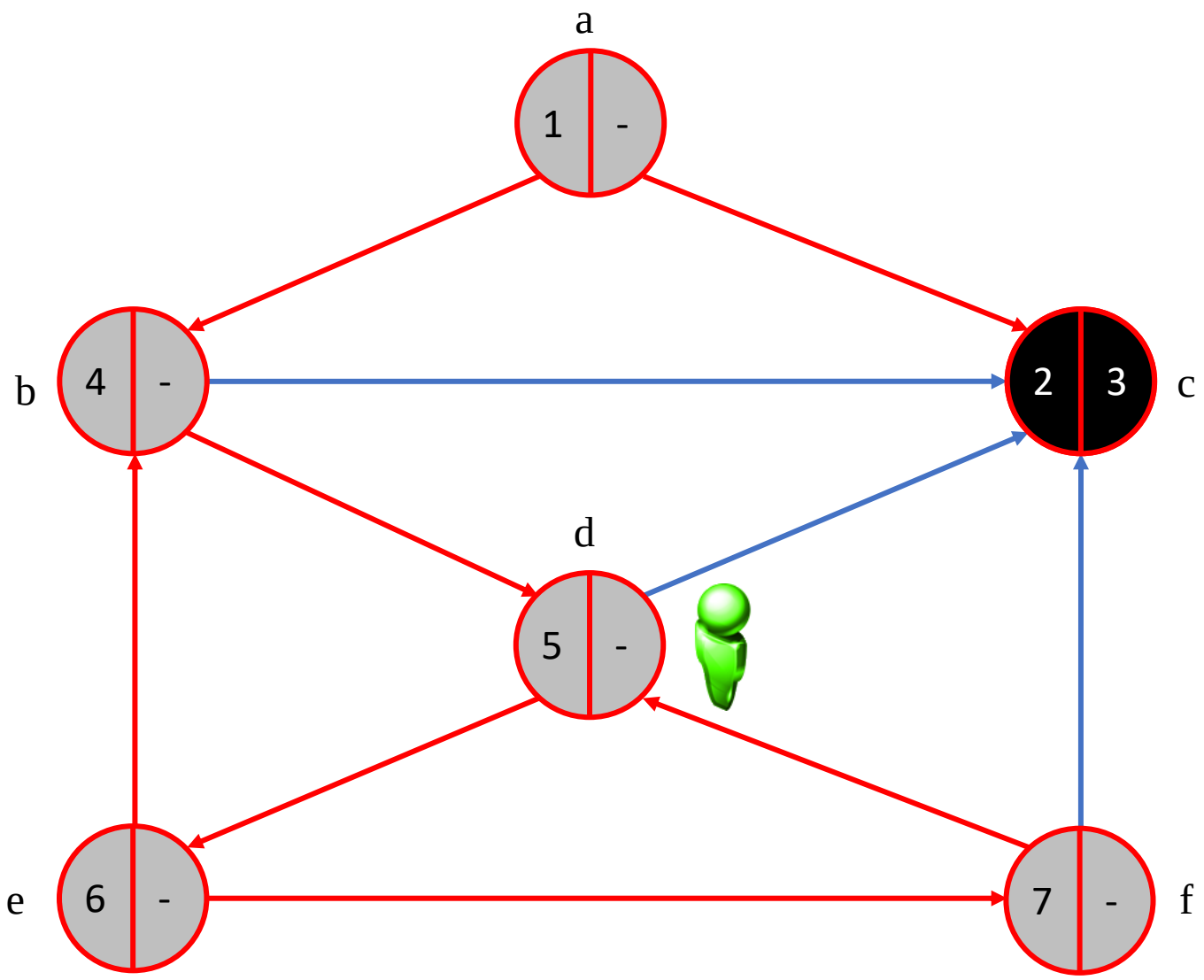
Kosaraju



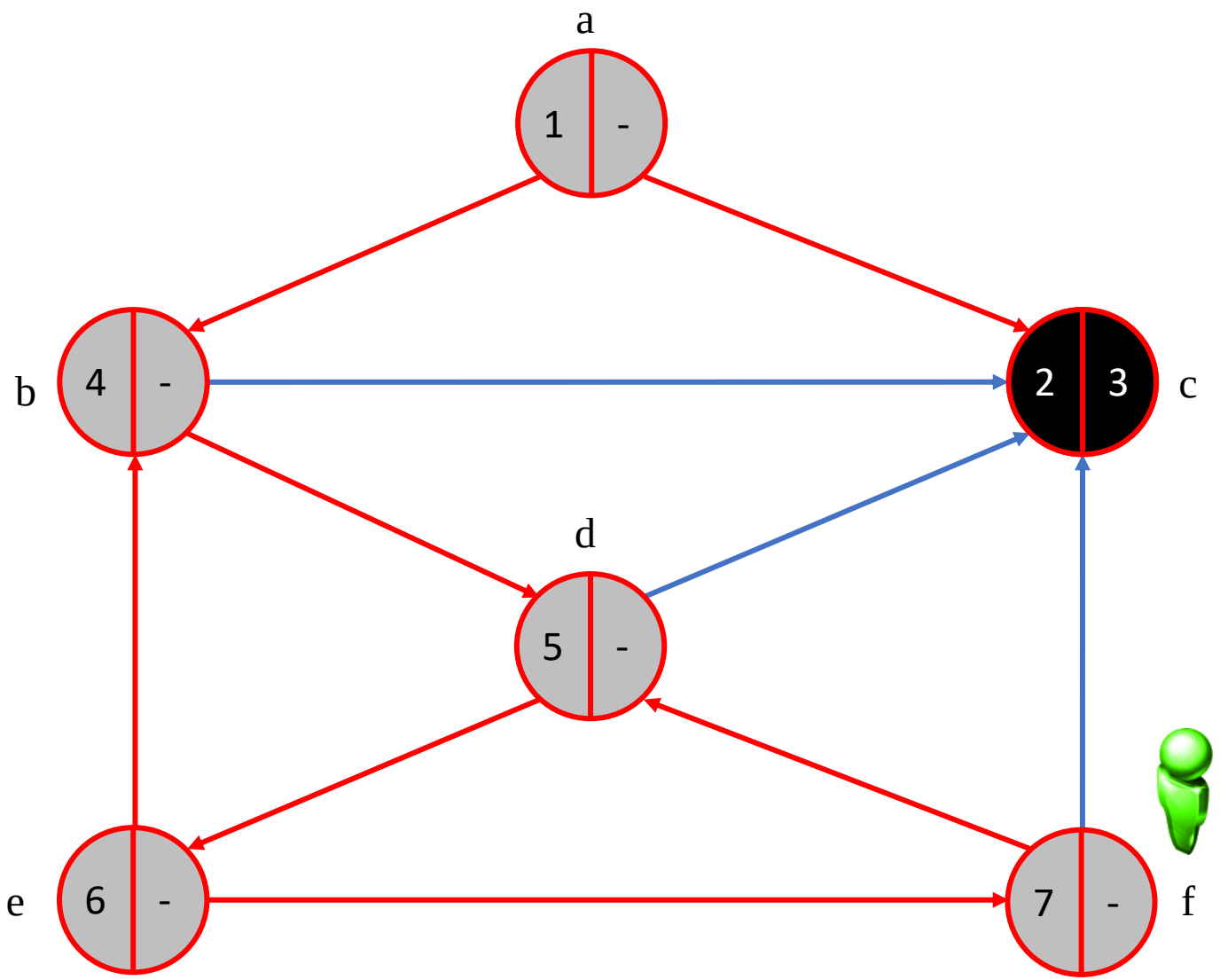
Kosaraju



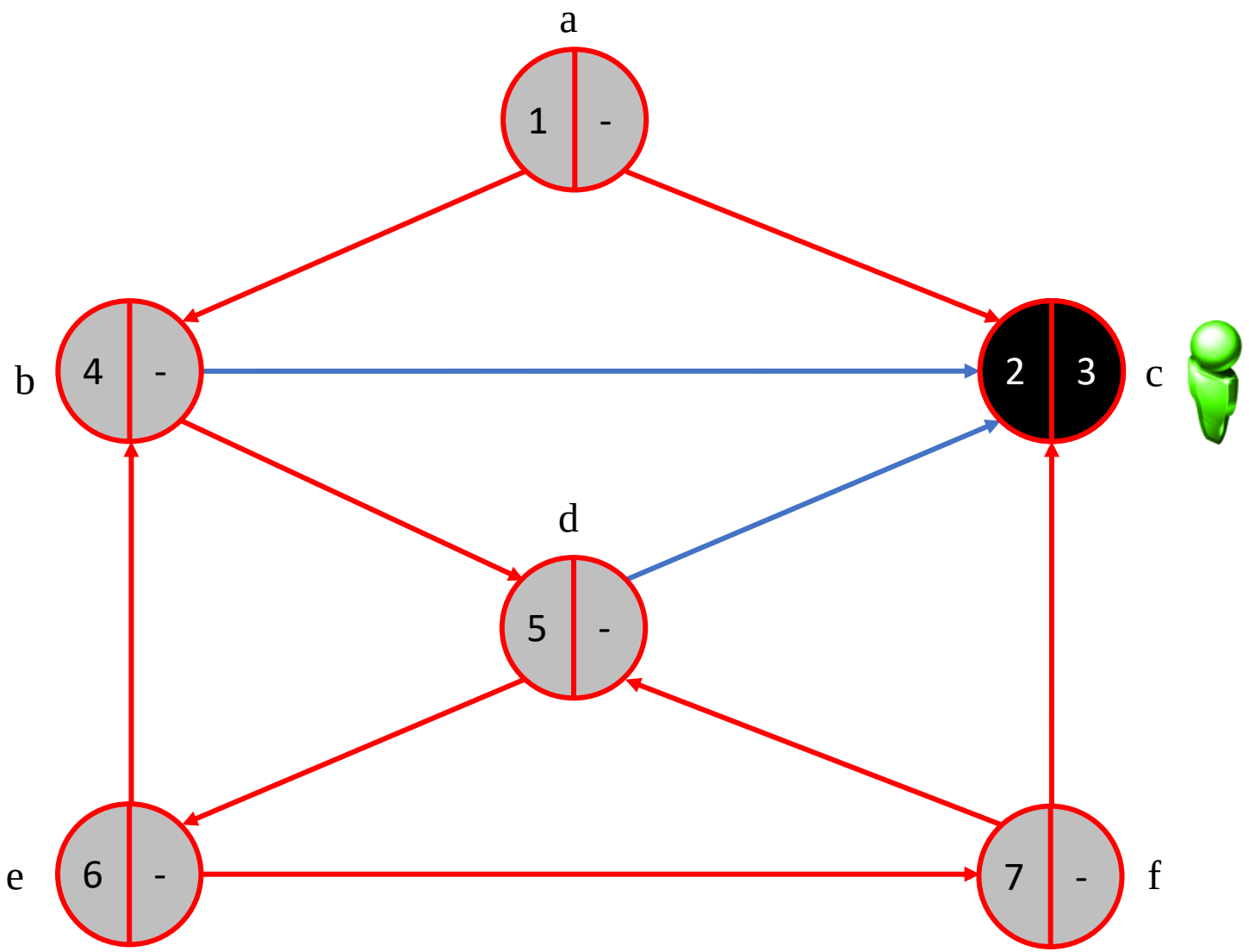
Kosaraju



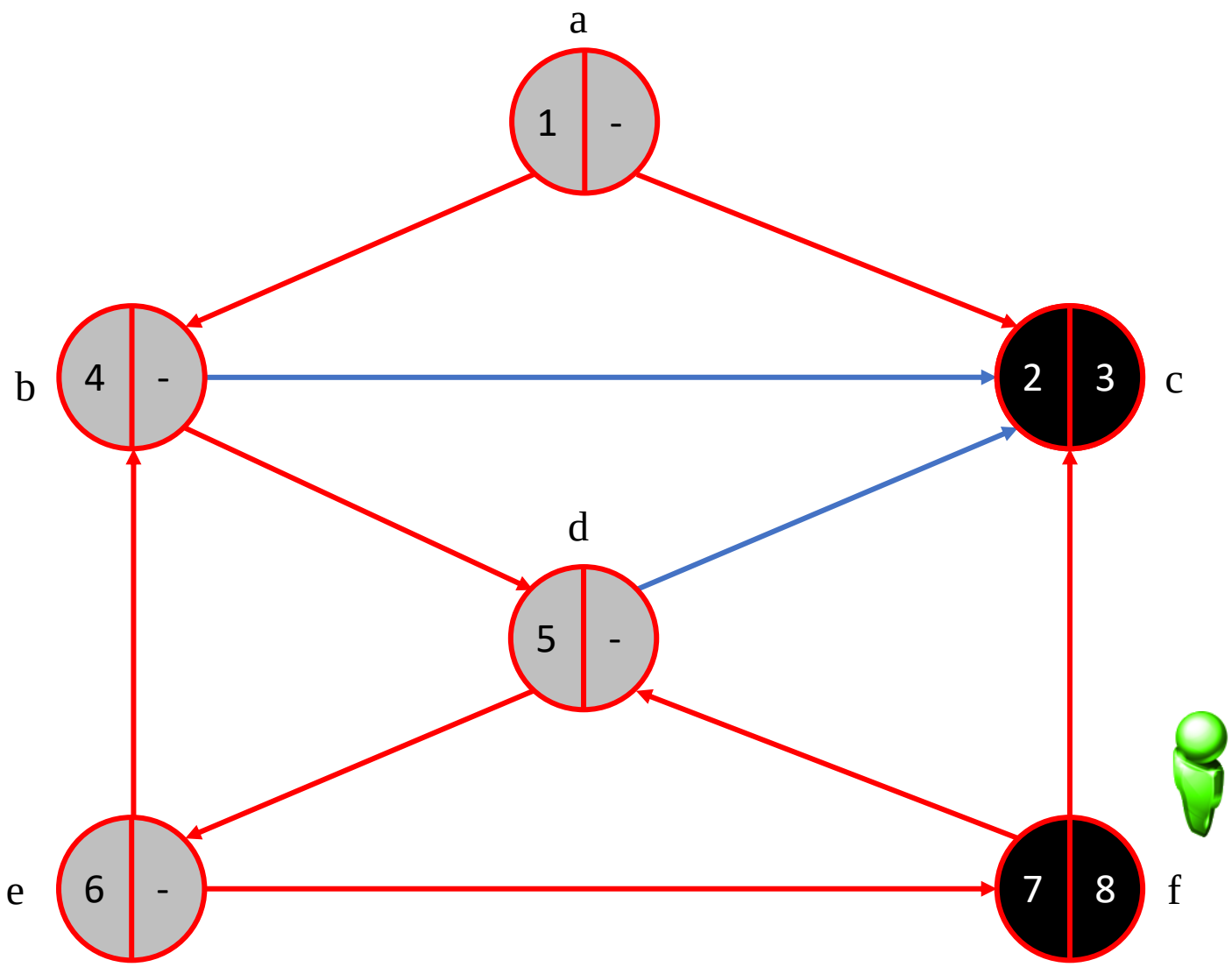
Kosaraju



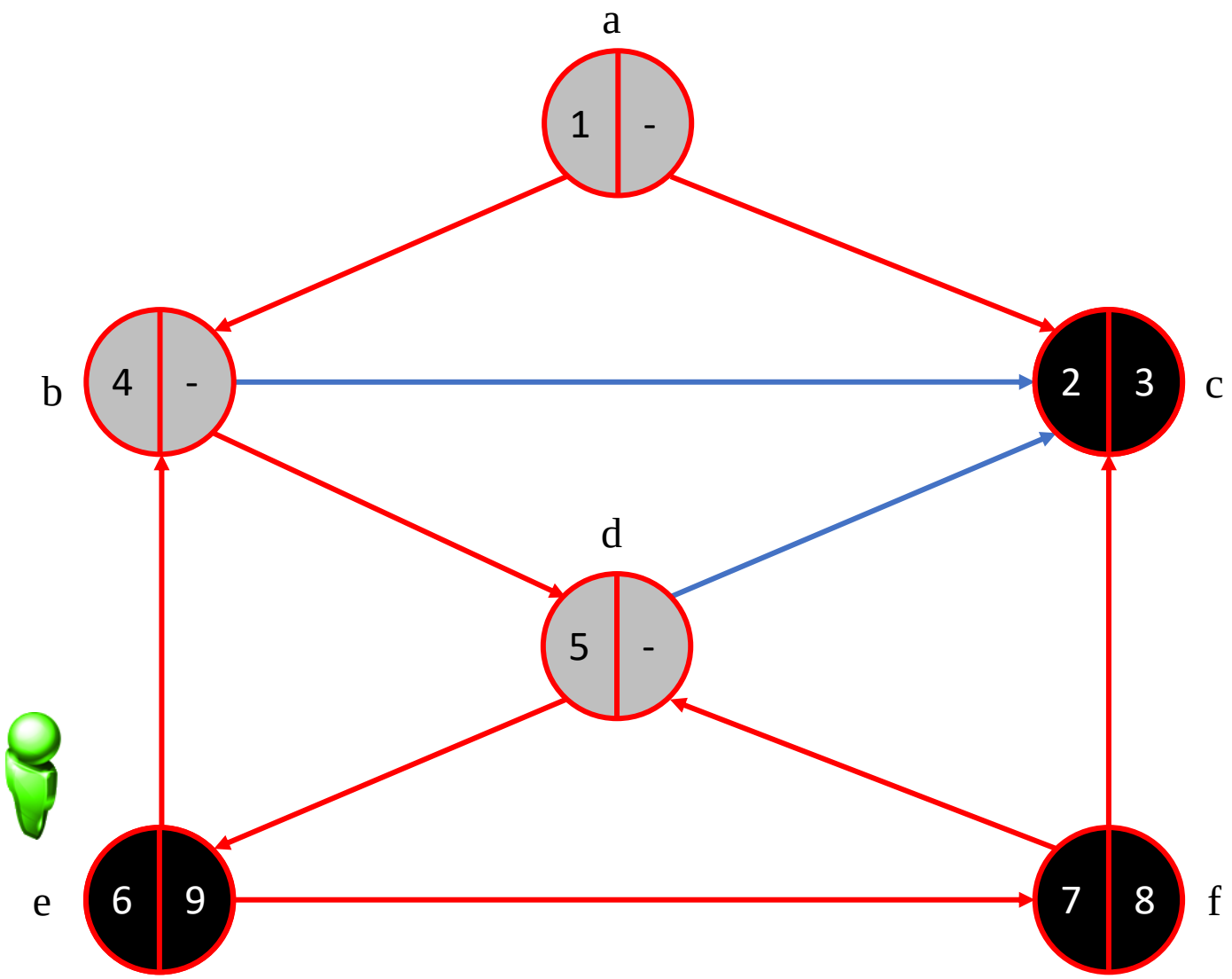
Kosaraju



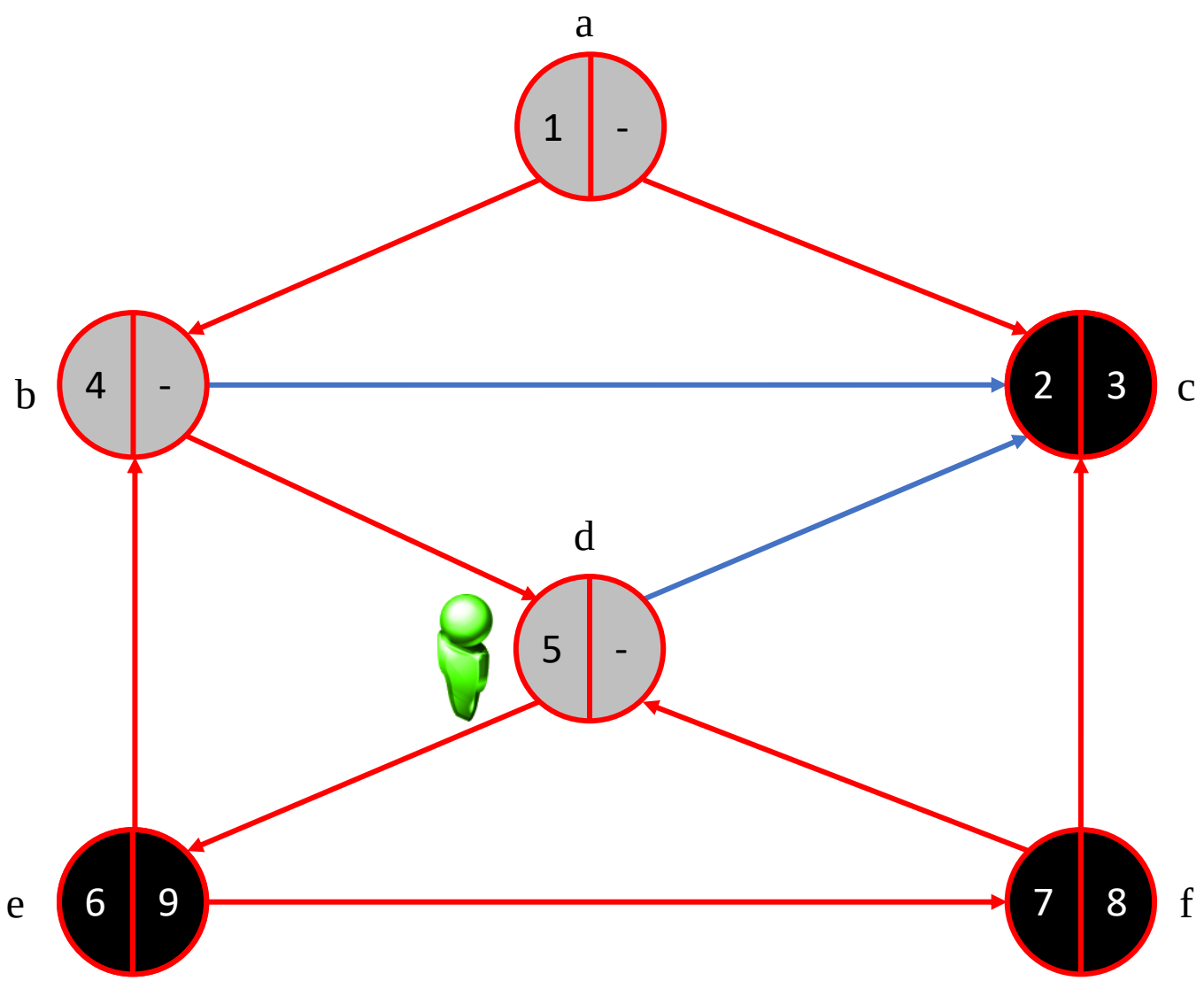
Kosaraju



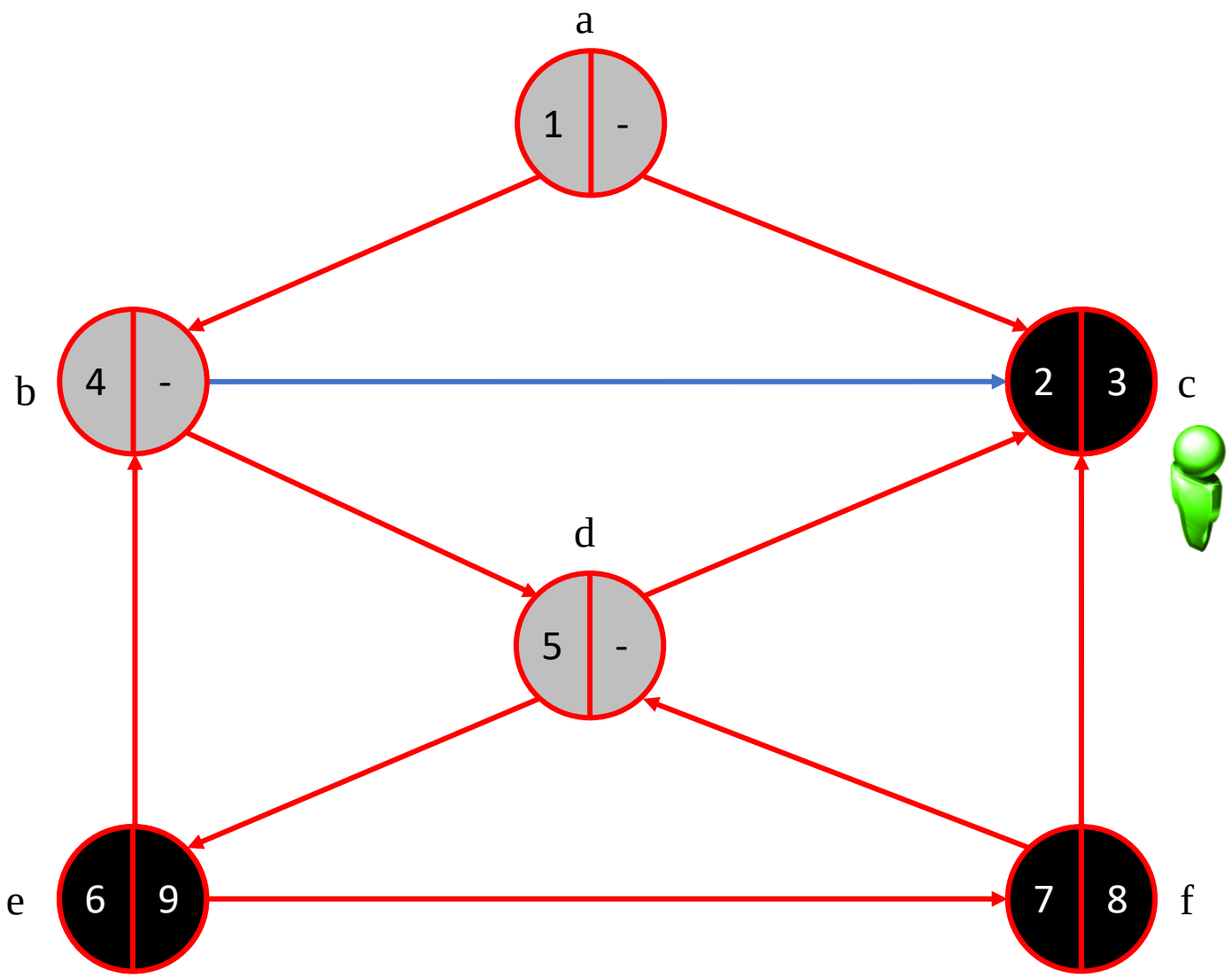
Kosaraju



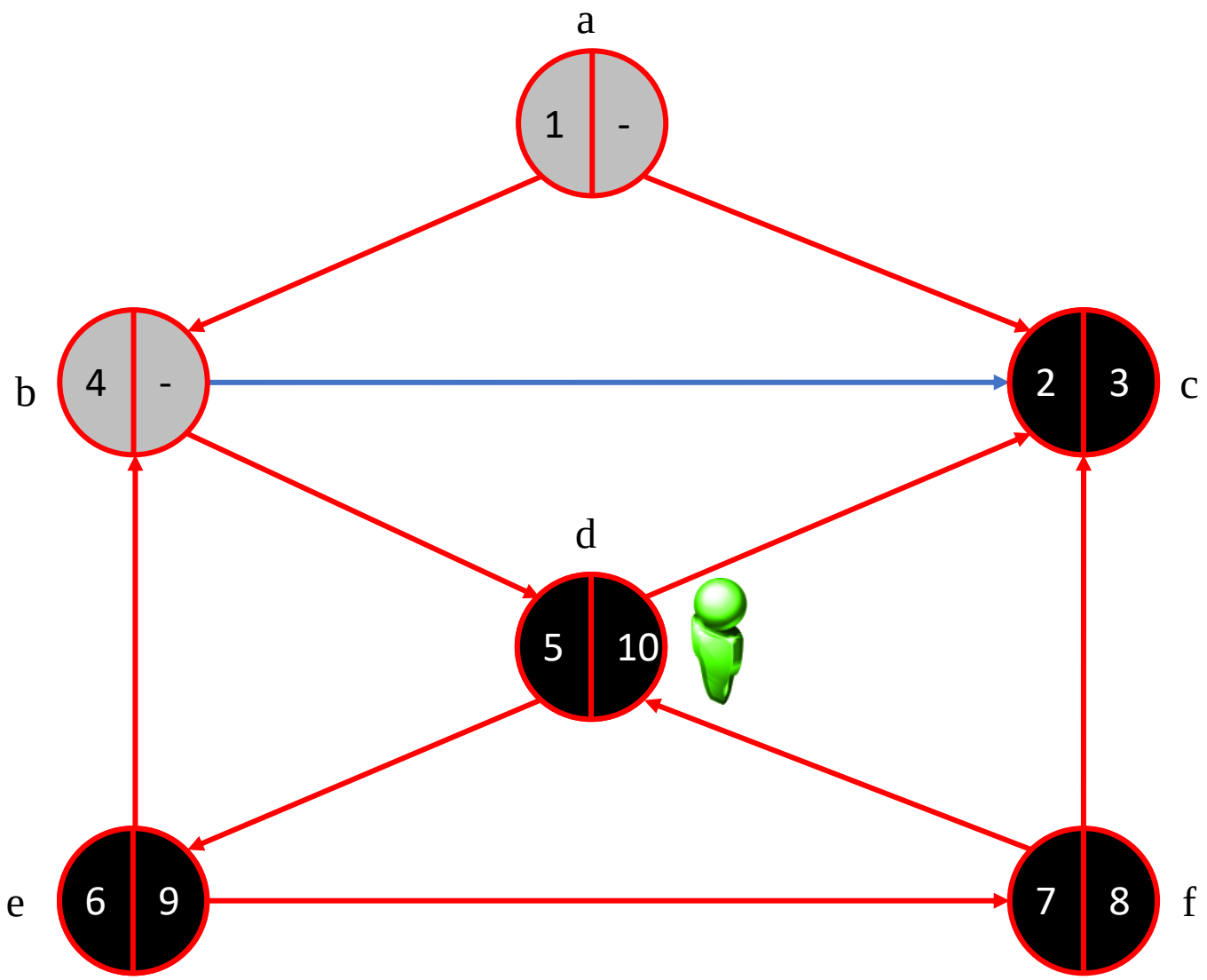
Kosaraju



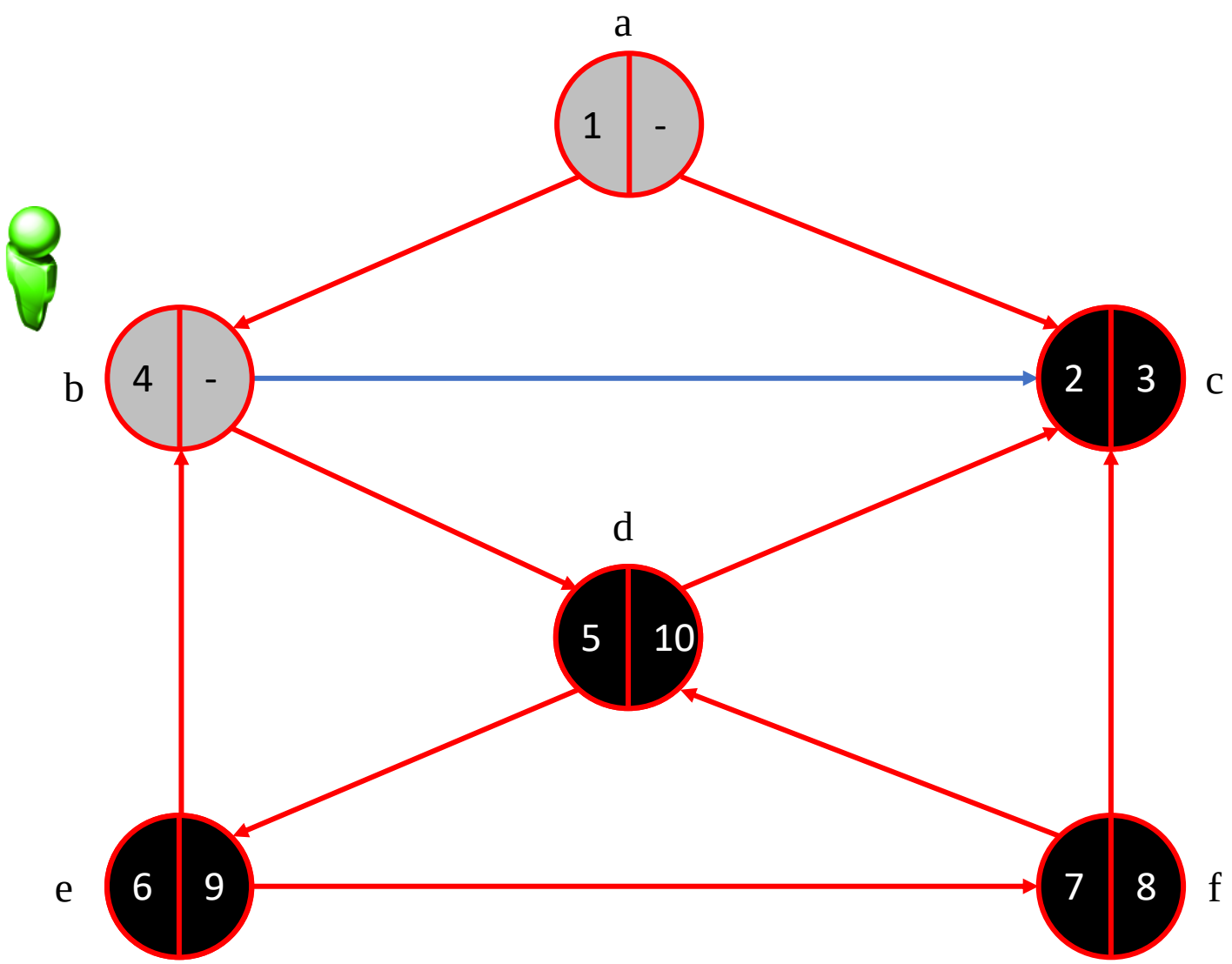
Kosaraju



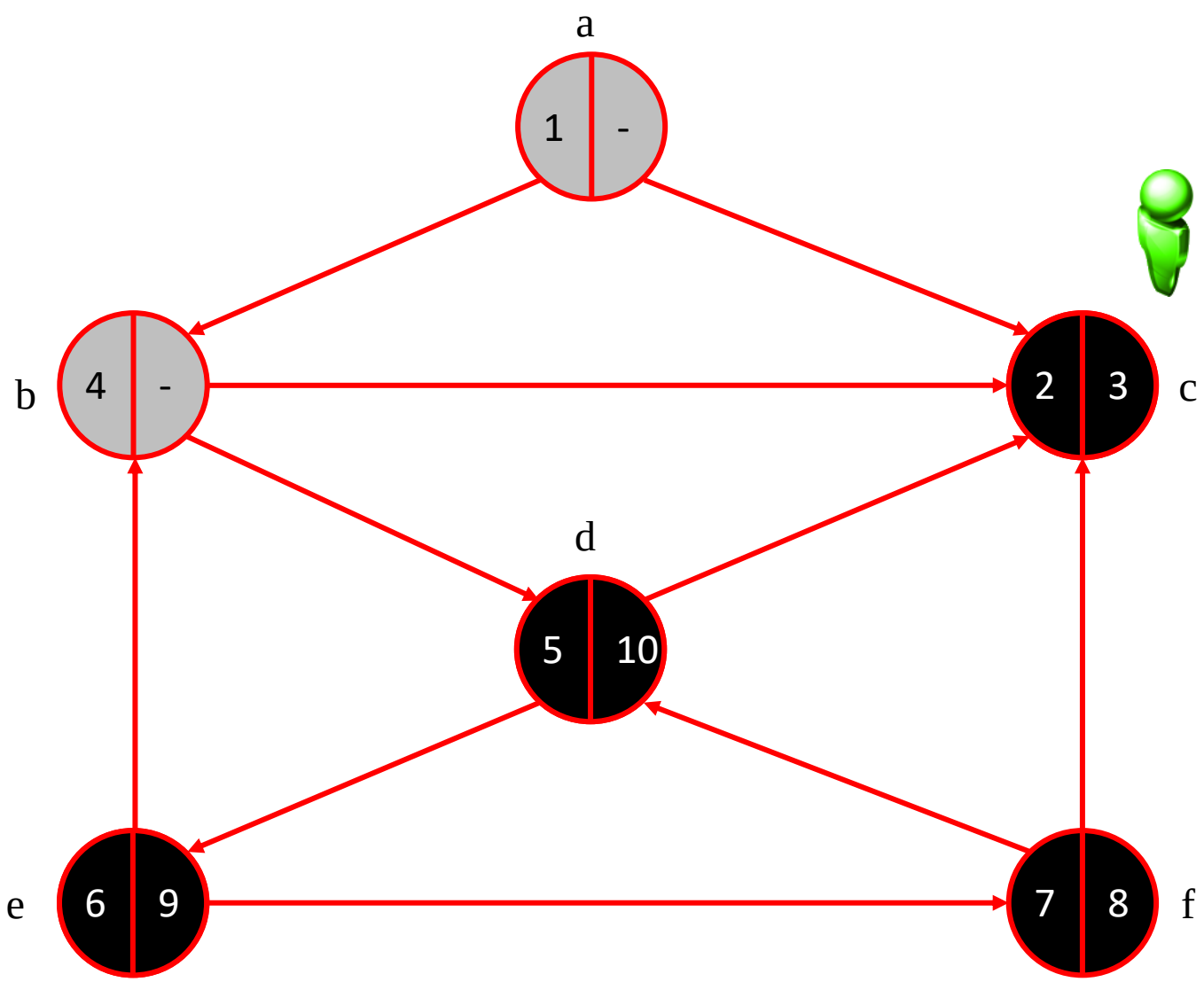
Kosaraju



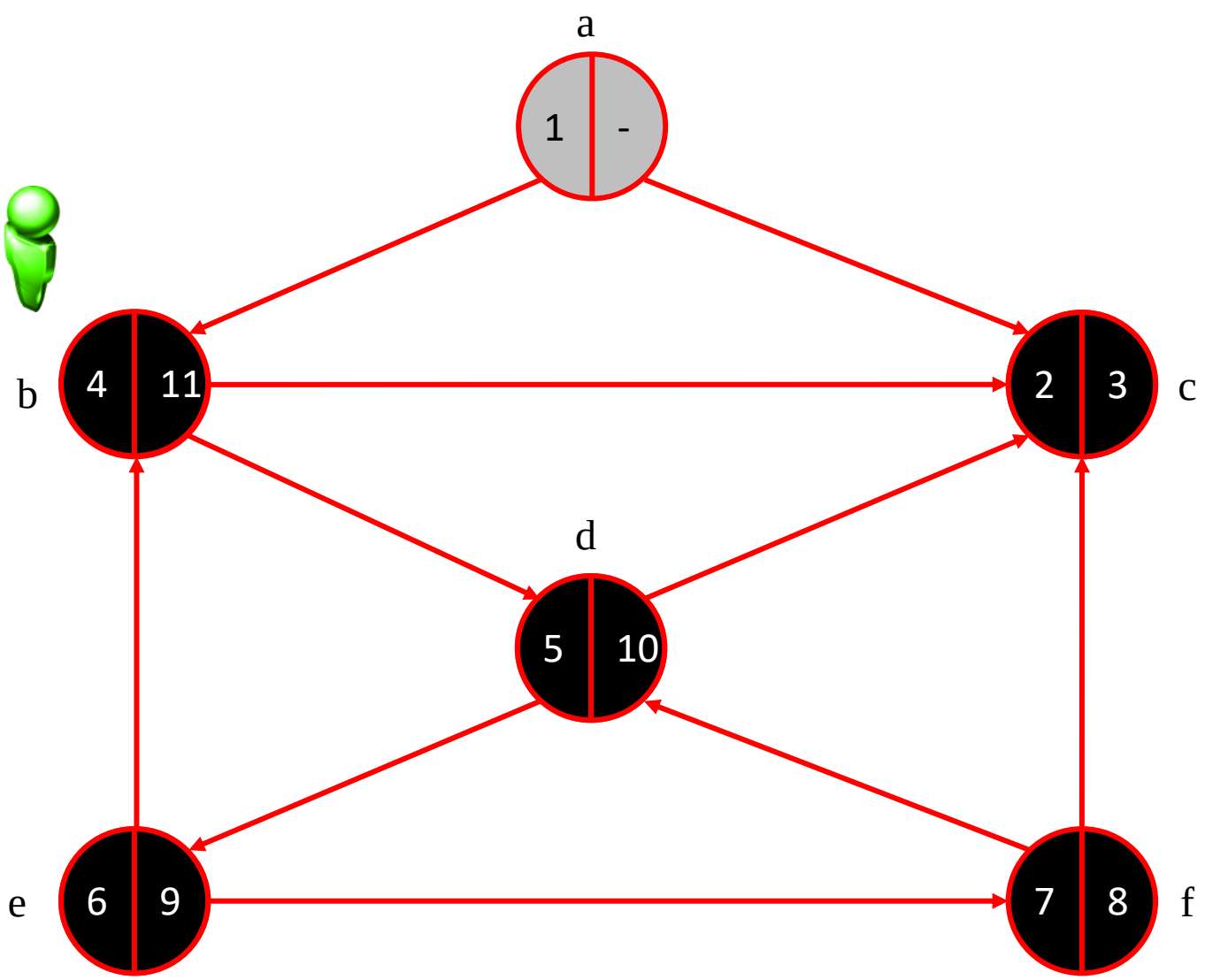
Kosaraju



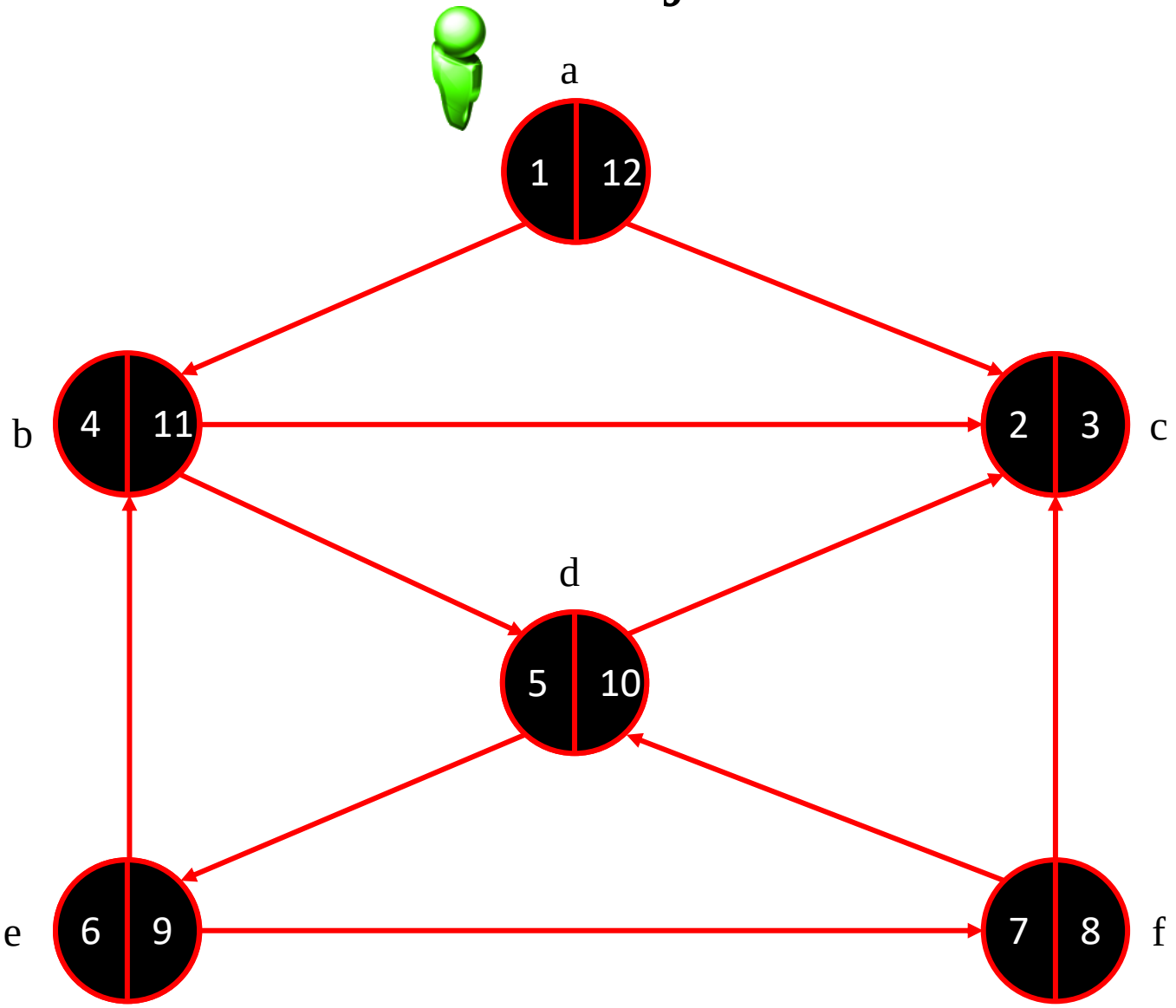
Kosaraju



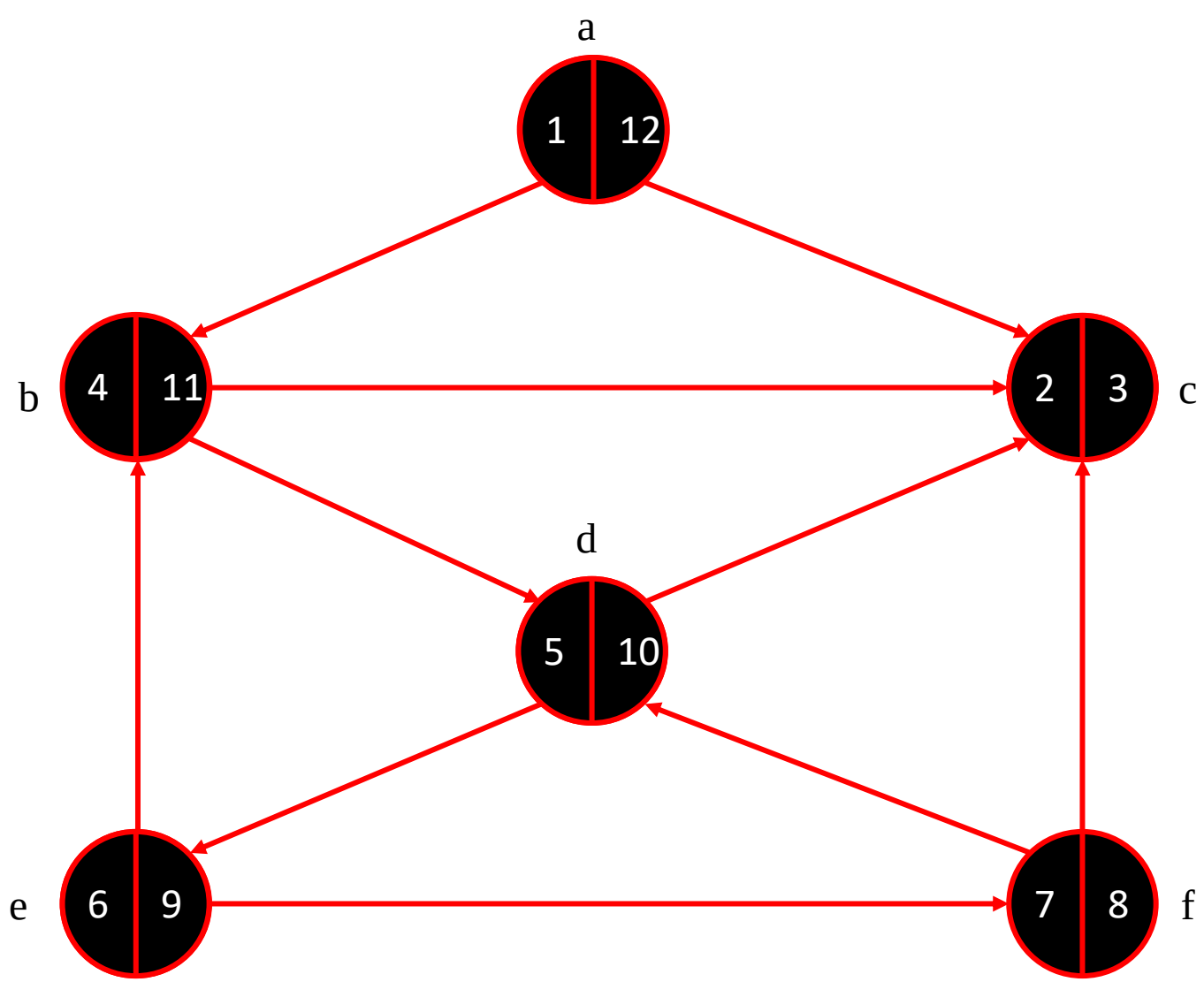
Kosaraju



Kosaraju



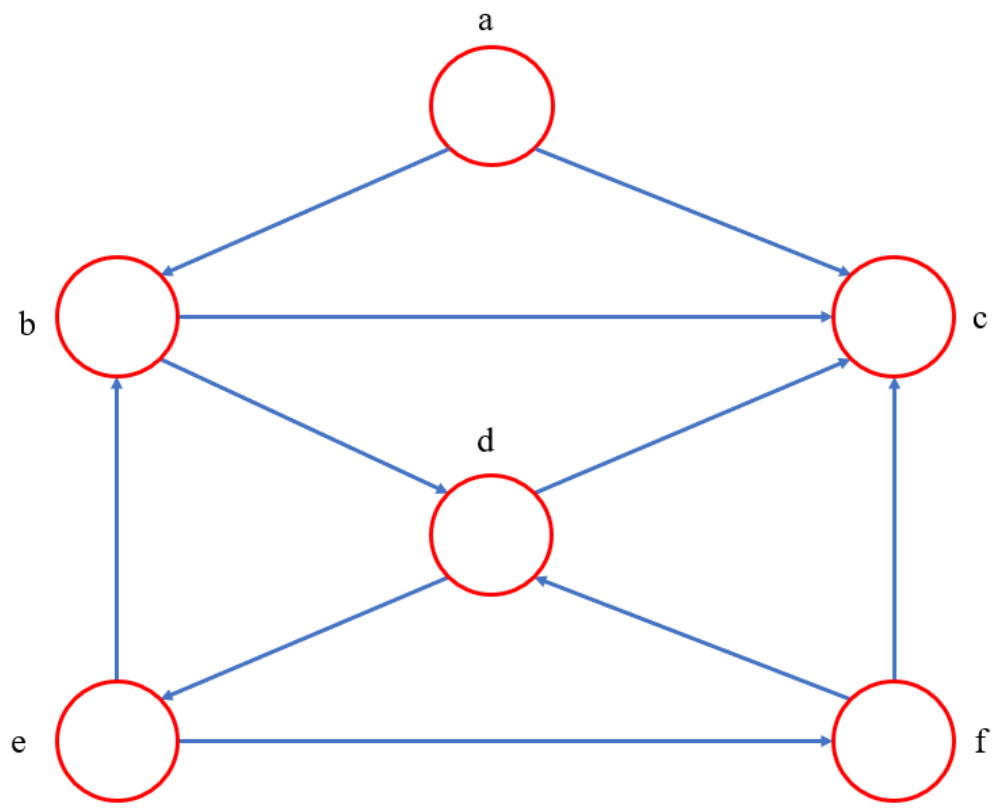
Kosaraju

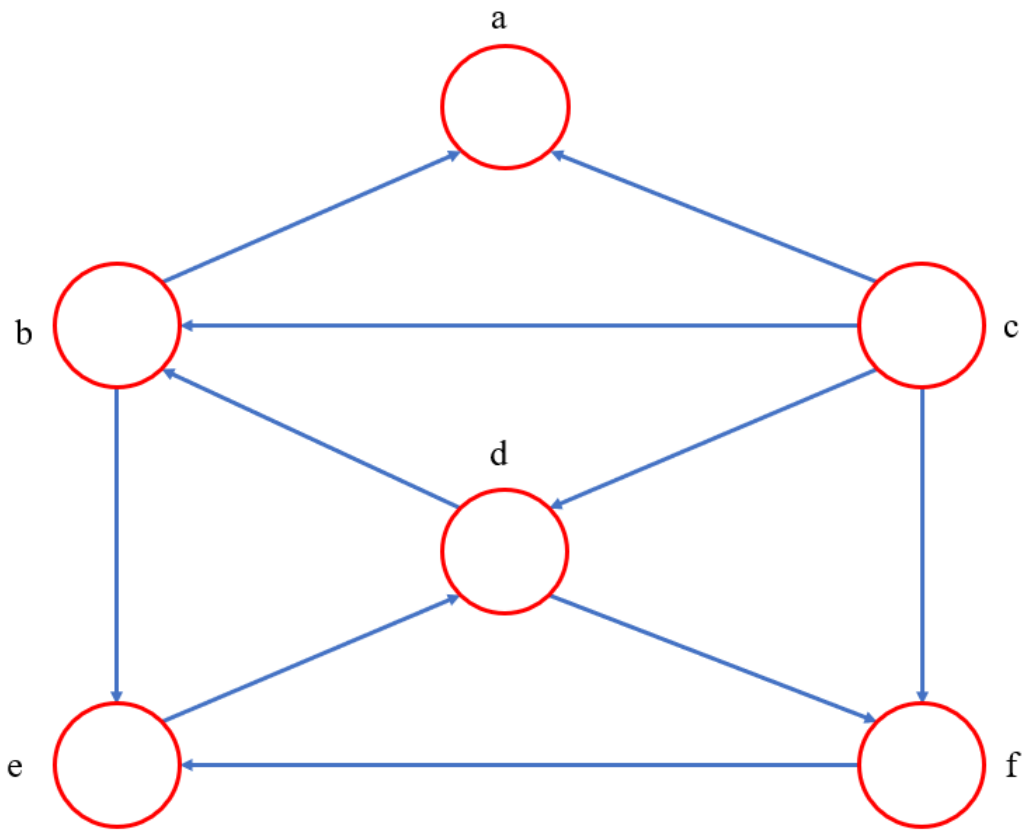


$\{c, f, e, d, b, a\}$

$\{a, b, d, e, f, c\}$

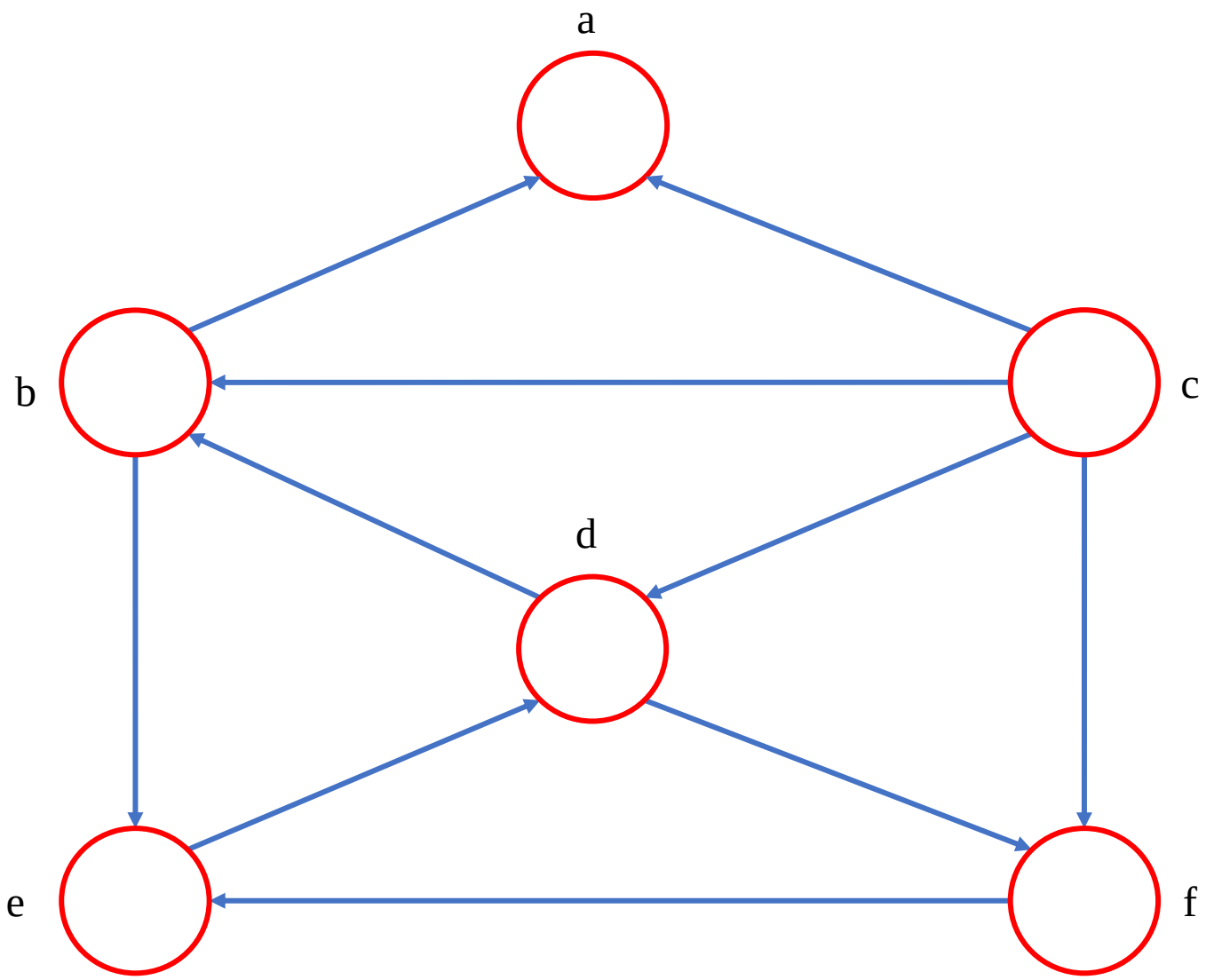
Kosaraju





Kosaraju

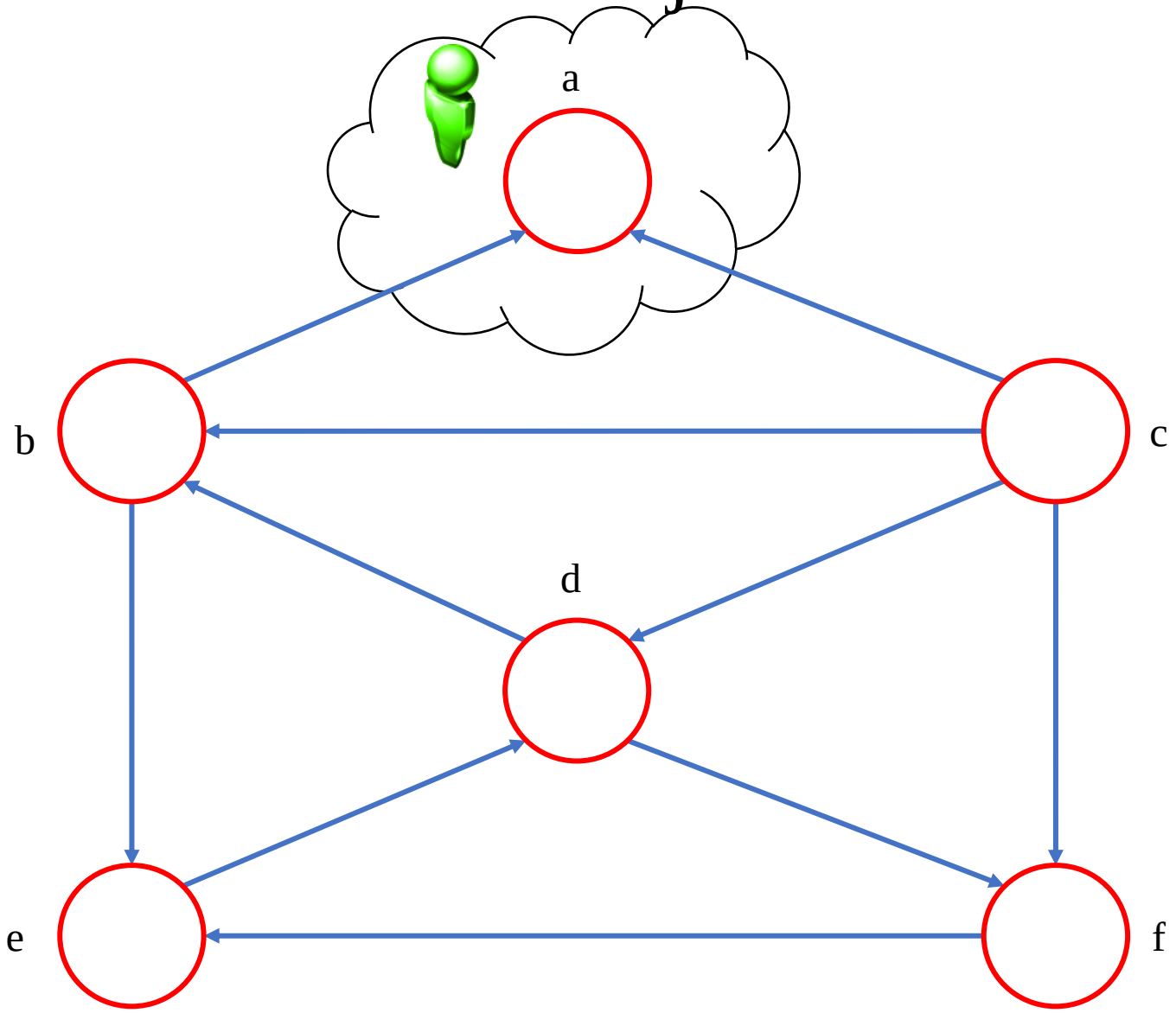
$\{a, b, d, e, f, c\}$



Kosaraju

SCCs

$\{a\}$

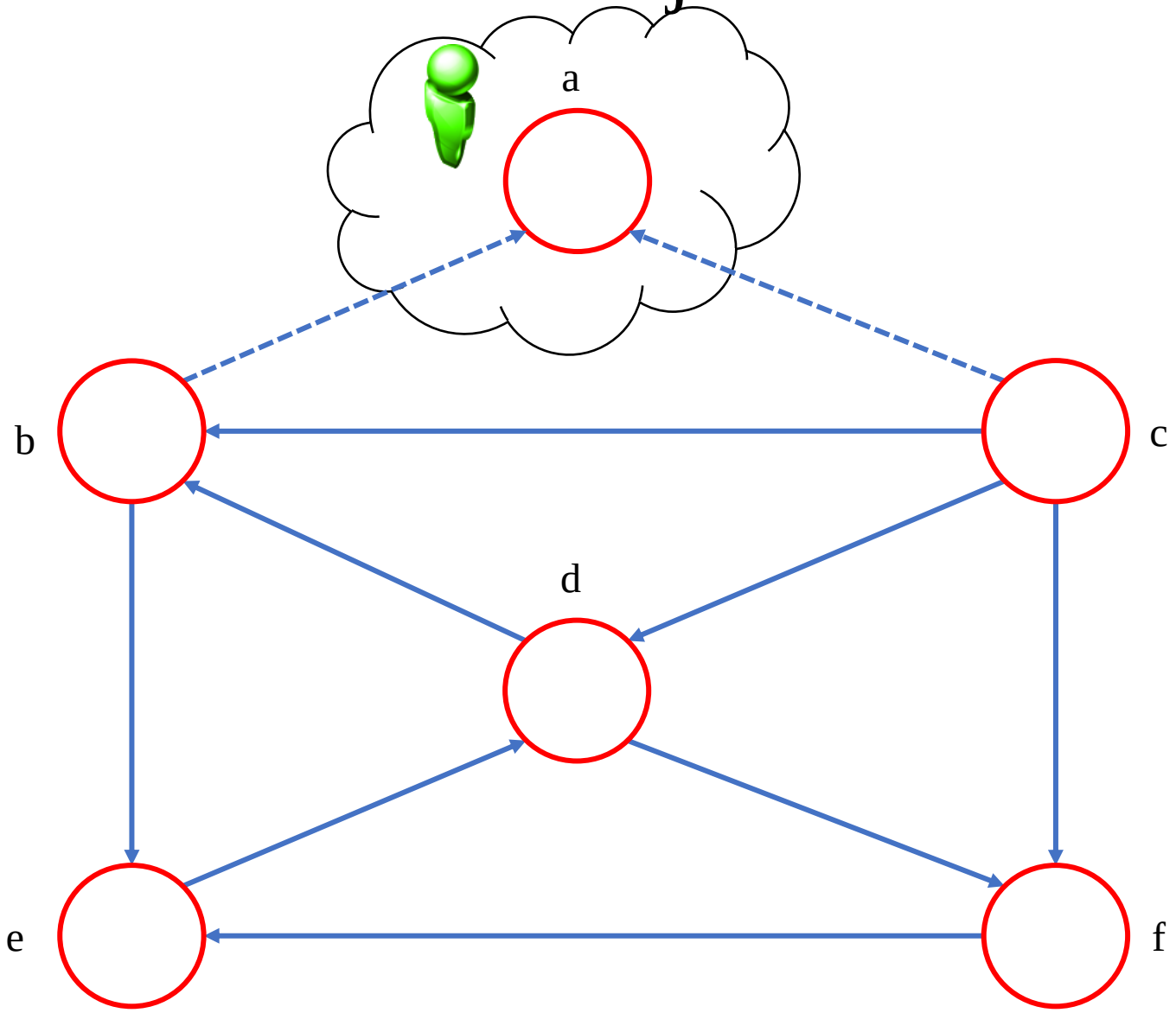


$\{a, b, d, e, f, c\}$

Kosaraju

SCCs

$\{a\}$



$\{a, b, d, e, f, c\}$

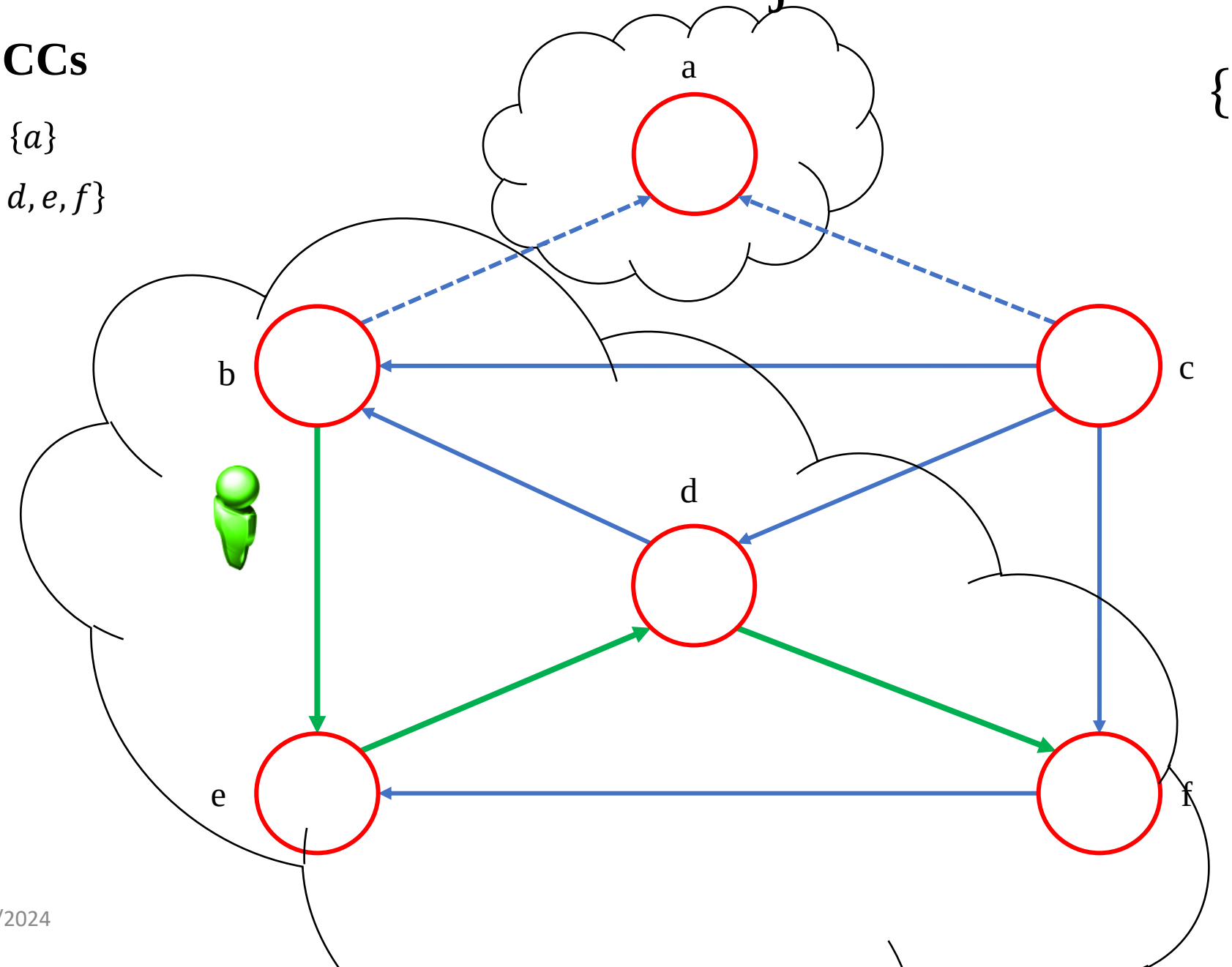
Kosaraju

SCCs

$\{a\}$

$\{b, d, e, f\}$

$\{a, b, d, e, f, c\}$



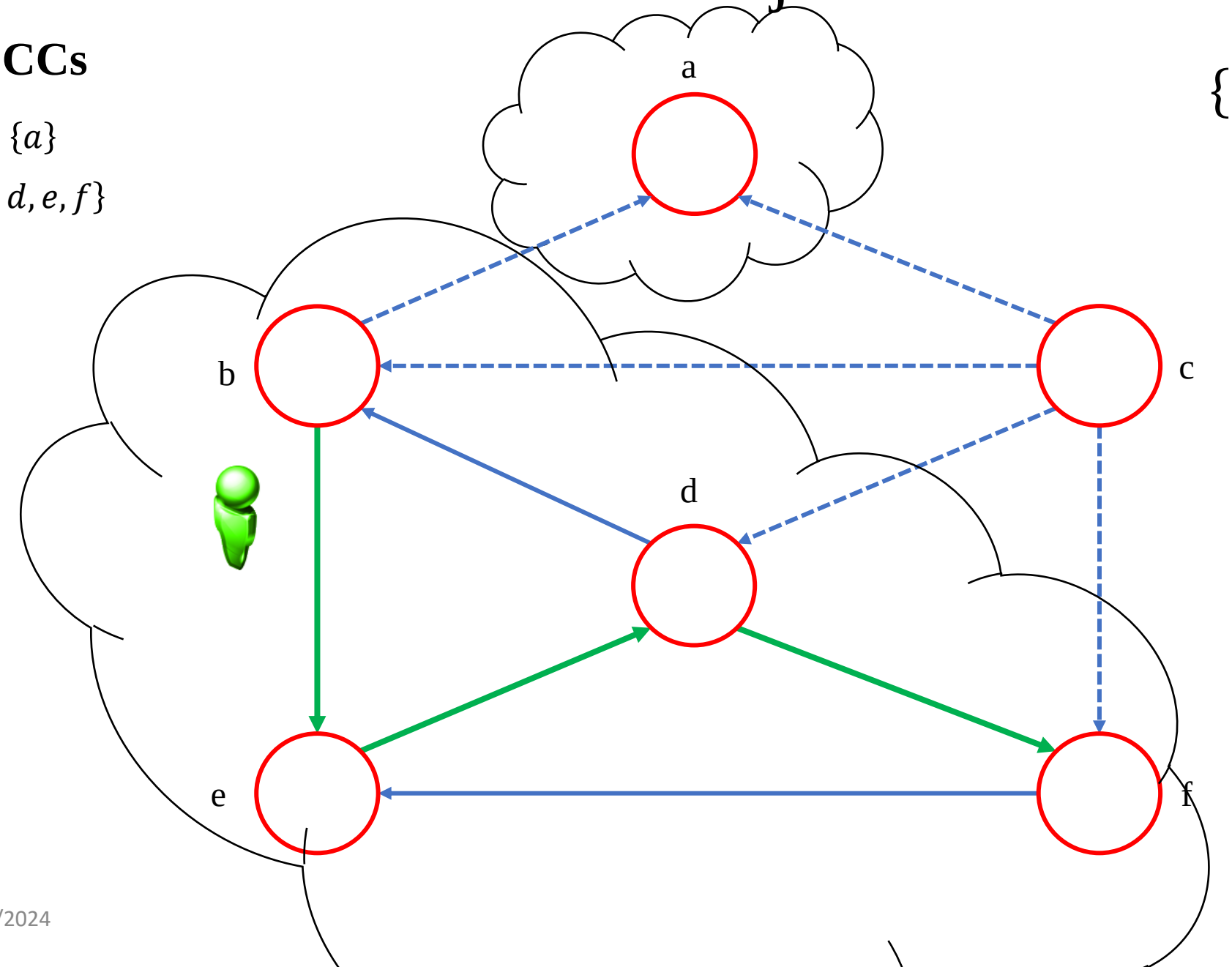
Kosaraju

SCCs

$\{a\}$

$\{b, d, e, f\}$

$\{a, b, d, e, f, c\}$



Kosaraju

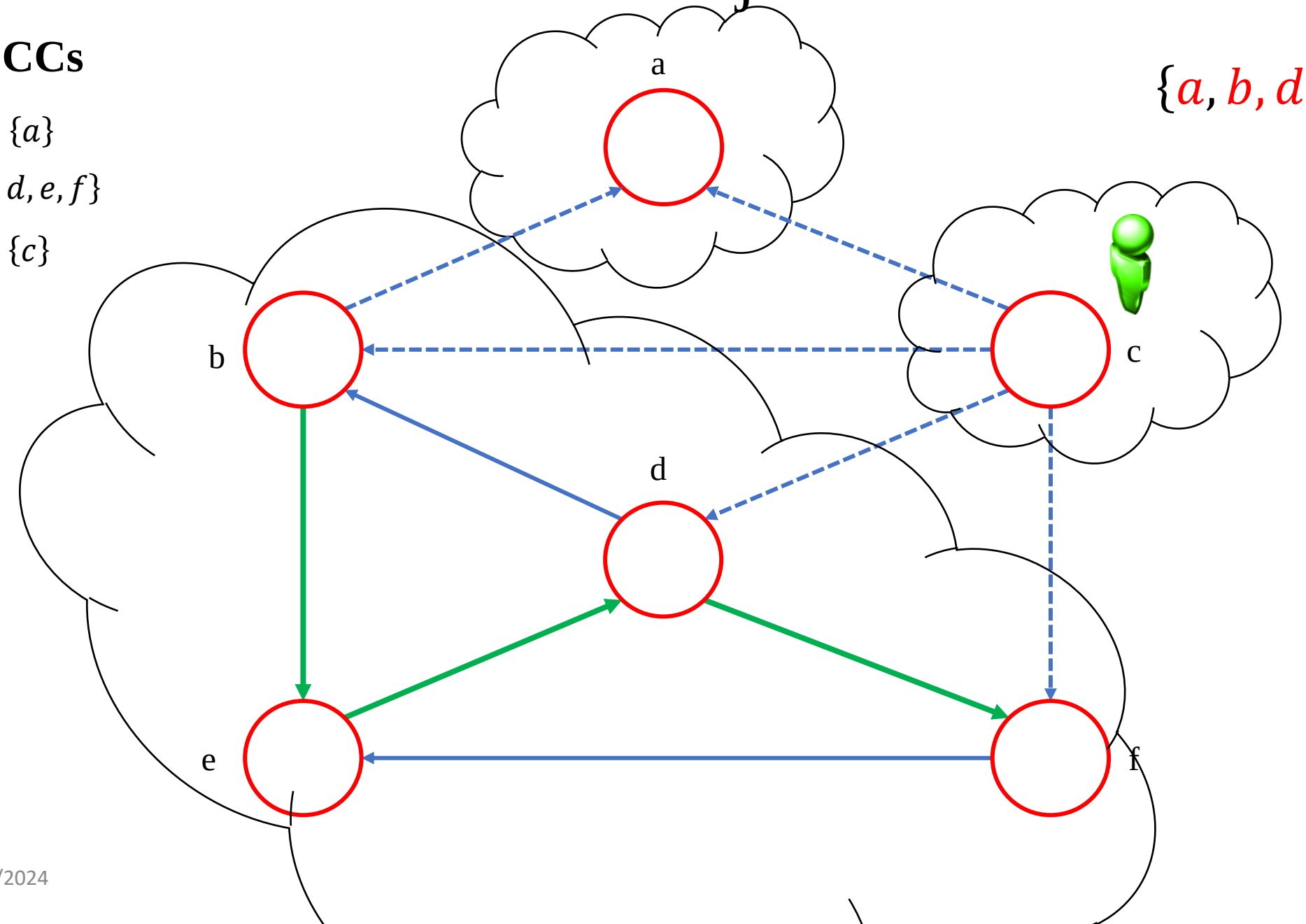
SCCs

$\{a\}$

$\{b, d, e, f\}$

$\{c\}$

$\{a, b, d, e, f, c\}$



Kosaraju

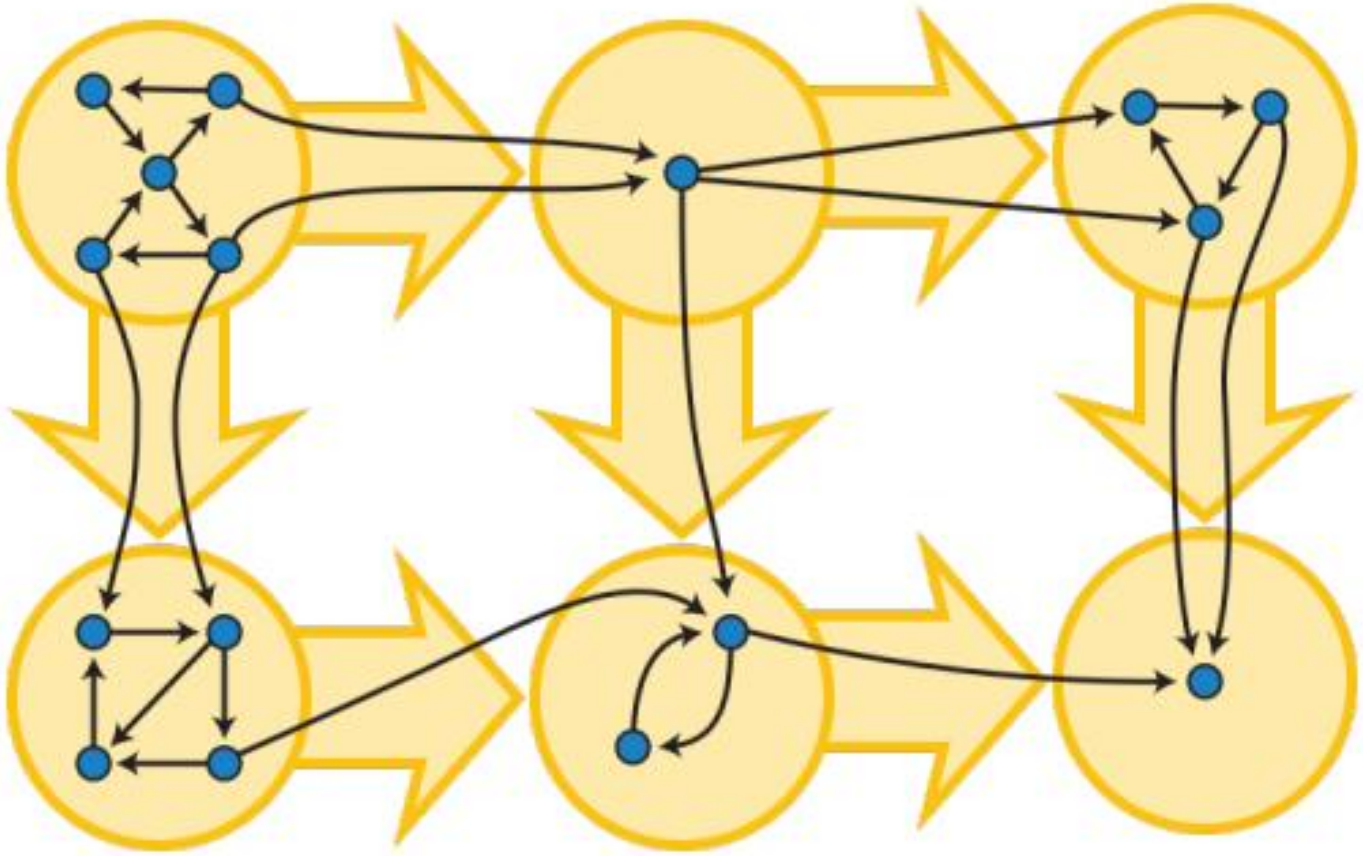
```
vector<vector<int>> adj, adj_rev;
vector<bool> used;
vector<int> order, component;

void dfs1(int v) {
    used[v] = true;

    for (auto u : adj[v])
        if (!used[u])
            dfs1(u);

    order.pb(v);
}

void dfs2(int v) {
    used[v] = true;
    component.pb(v);

    for (auto u : adj_rev[v])
        if (!used[u])
            dfs2(u);
}
```

```
int main() {
    int n,m; cin>>n>>m;
    adj.resize(n); adj_rev.resize(n);
    for(int i = 0; i < m; i++) {
        int a, b; cin>>a>>b;
        a--; b--;
        adj[a].pb(b);
        adj_rev[b].pb(a);
    }
    used.assign(n, false);
    for (int i = 0; i < n; i++){
        if (!used[i])
            dfs1(i);
    }
    used.assign(n, false);
    reverse(order.begin(), order.end());
    for (auto v : order){
        if (!used[v]) {
            dfs2 (v);
            for(int i = 0; i < sz(component); i++) cout<<component[i]+1<<" ";
            cout<<endl;
            component.clear();
        }
    }
}
```

Problema Ejemplo

- https://codeforces.com/problemset/problem/427/C

Condensación



2-SAT

- SAT (Boolean satisfabiality problem) es el problema de asignar valores booleanos a variables con el fin de satisfacer cierta fórmula. En general dicha fórmula se da en forma normal conjuntiva.
- 2-SAT es una restricción de SAT en donde cada clausula tiene exactamente dos literales.

$$(a \vee \neg b) \wedge (\neg a \vee b) \wedge (\neg a \vee \neg b) \wedge (a \vee \neg c)$$

2-SAT

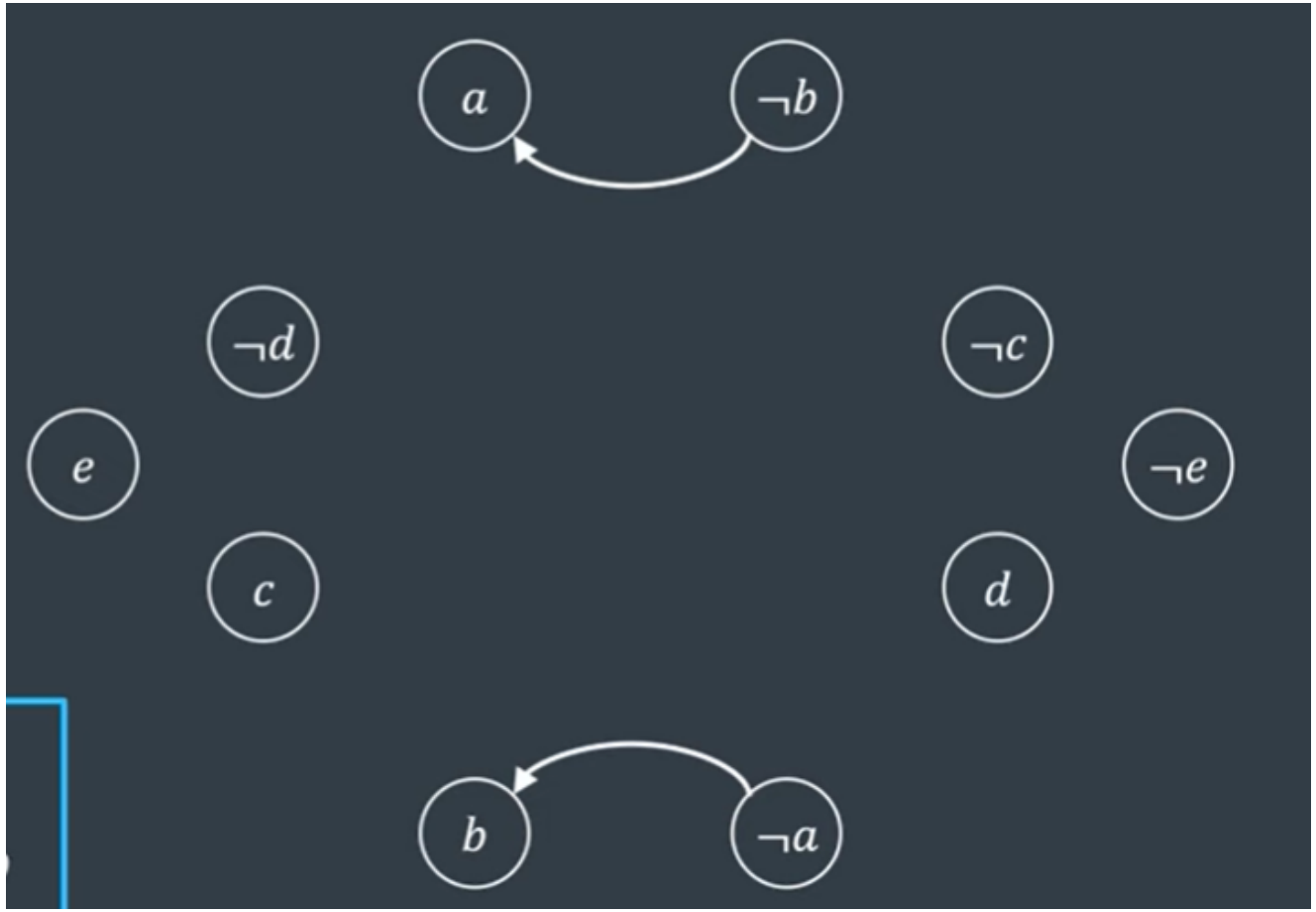
- Notemos que:

$$a \vee b \iff (\neg a \rightarrow b) \wedge (\neg b \rightarrow a)$$

2-SAT

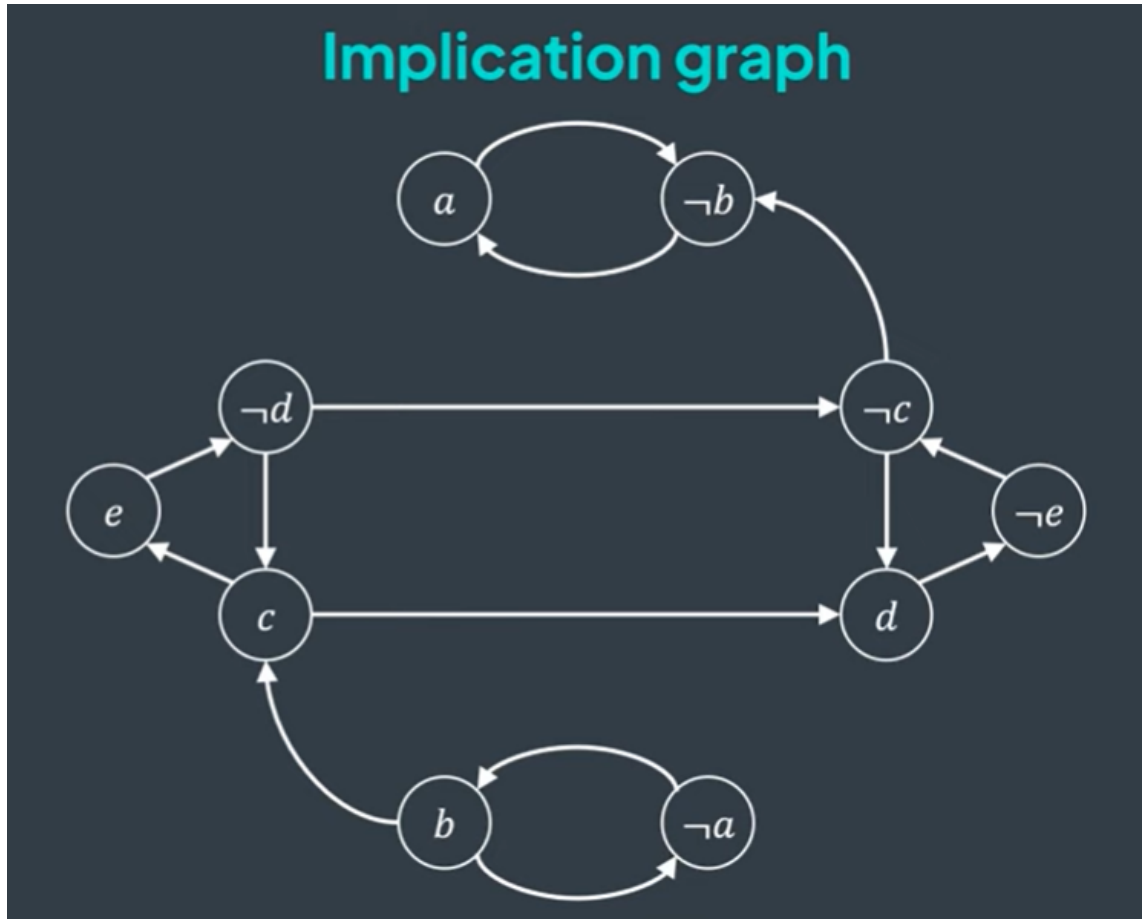
- $a \vee b$
- $\neg b \vee c$
- $\neg a \vee \neg b$
- $c \vee d$
- $\neg c \vee e$
- $\neg d \vee \neg e$
- $\neg c \vee d$

$$a \vee b \Leftrightarrow \neg a \longrightarrow b$$
$$\Leftrightarrow \neg b \longrightarrow a$$

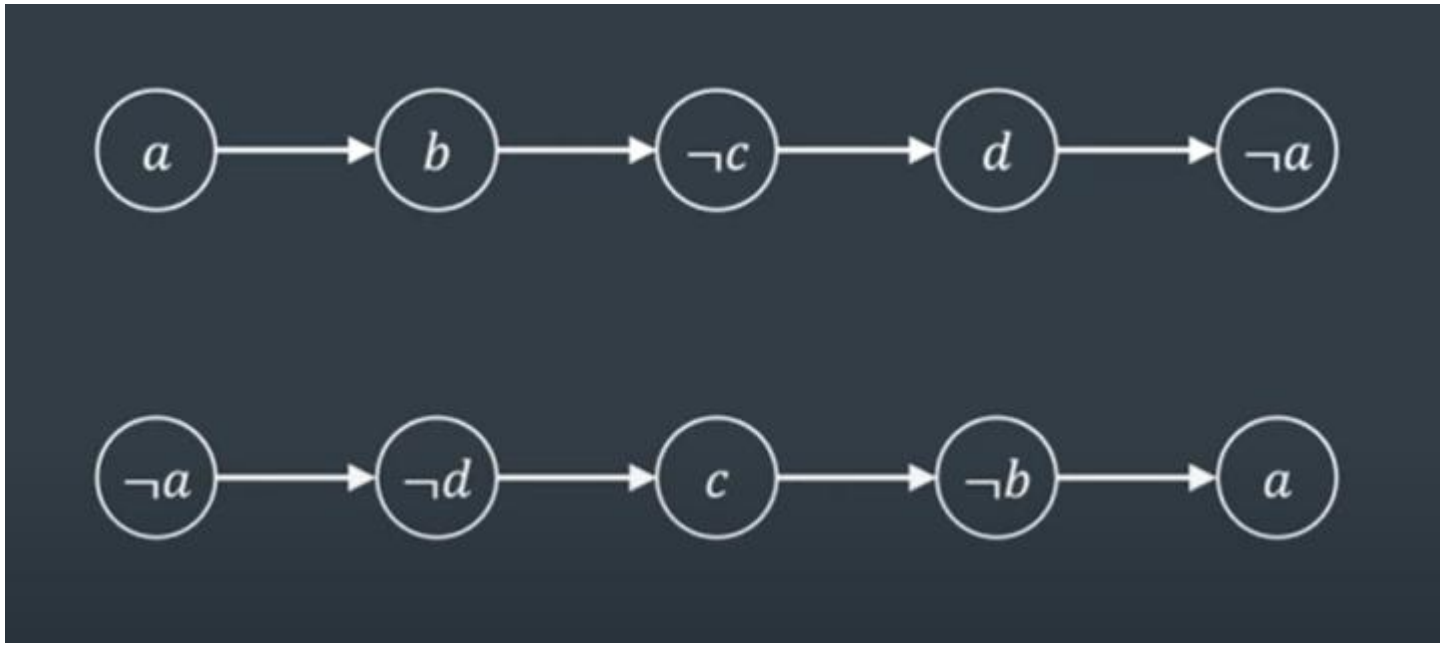


2-SAT

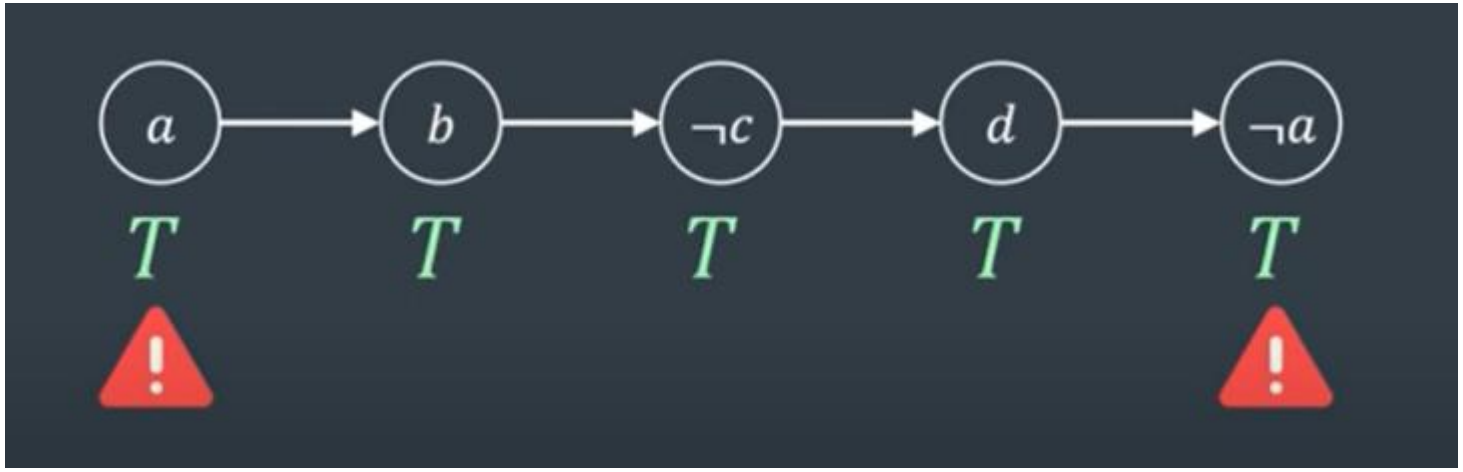
- $a \vee b$
- $\neg b \vee c$
- $\neg a \vee \neg b$
- $c \vee d$
- $\neg c \vee e$
- $\neg d \vee \neg e$
- $\neg c \vee d$

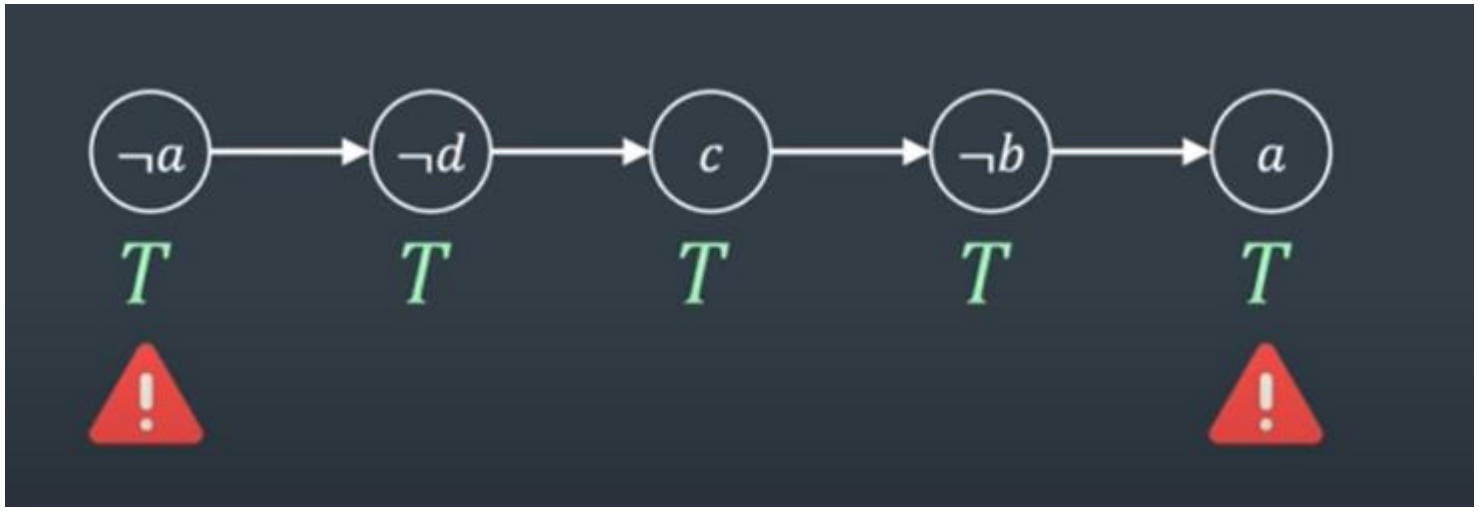


2-SAT



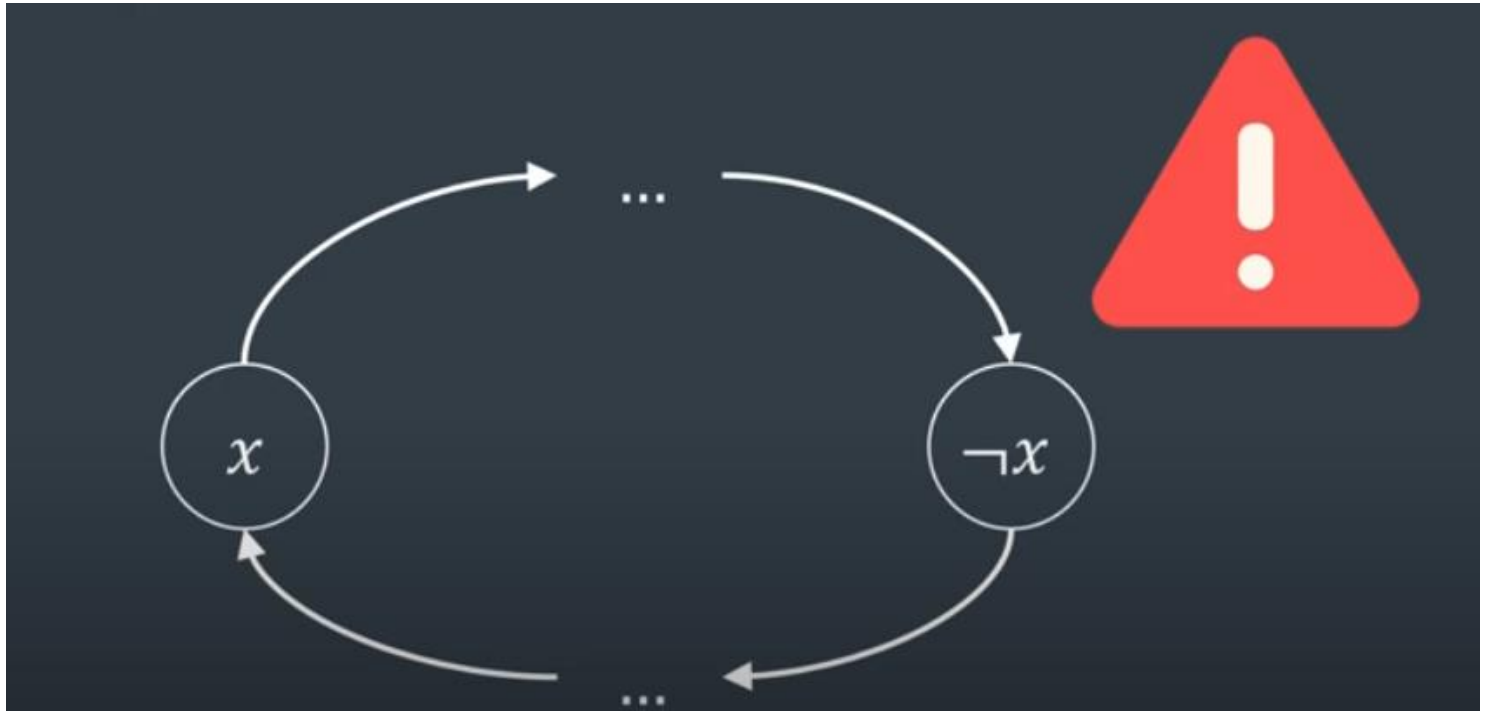
2-SAT





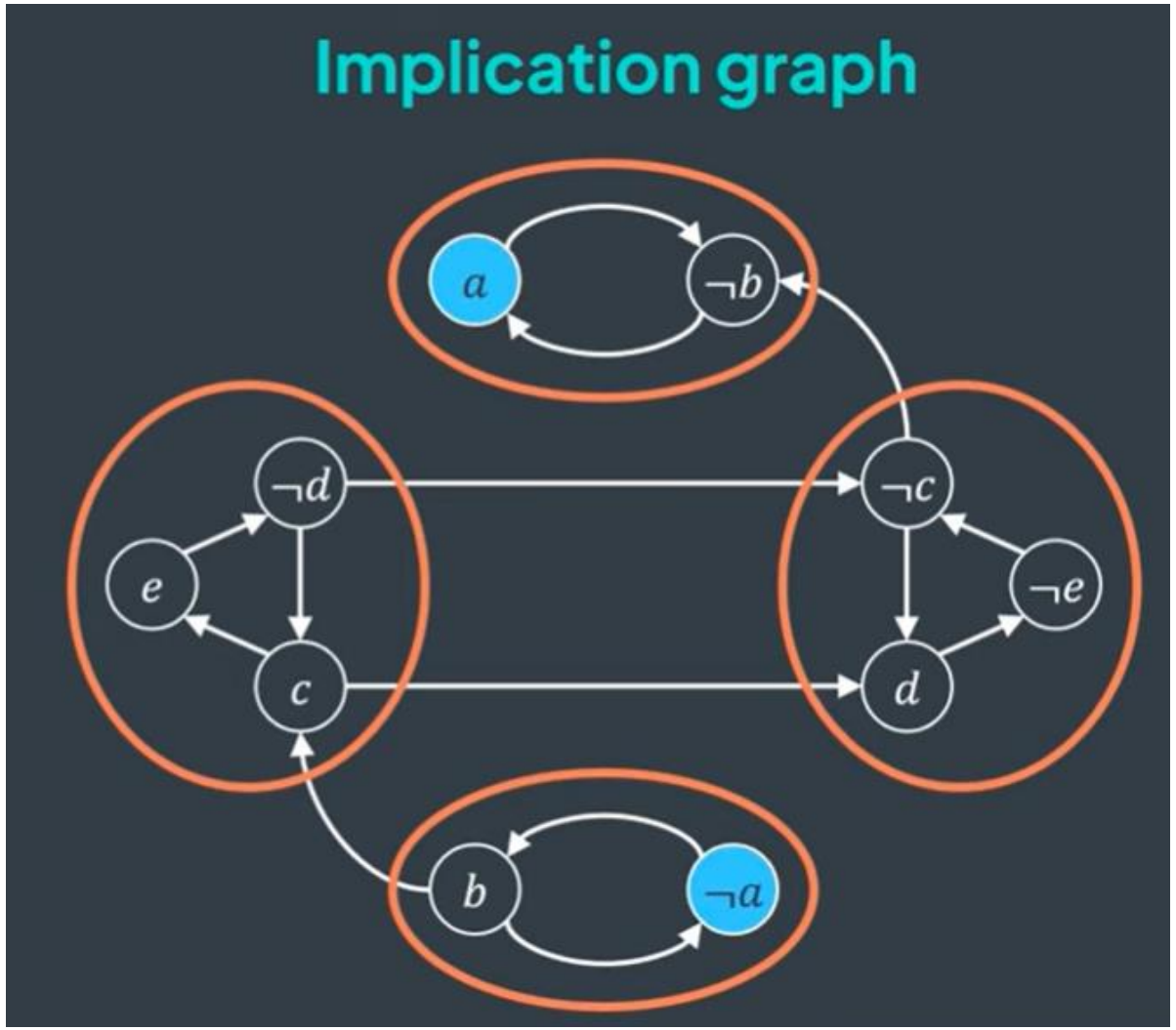
2-SAT

- Si para una variable x, tenemos en el grafo de implicación un camino de x a ¬x y un camino de ¬x a x, la fórmula no es satisfacible.



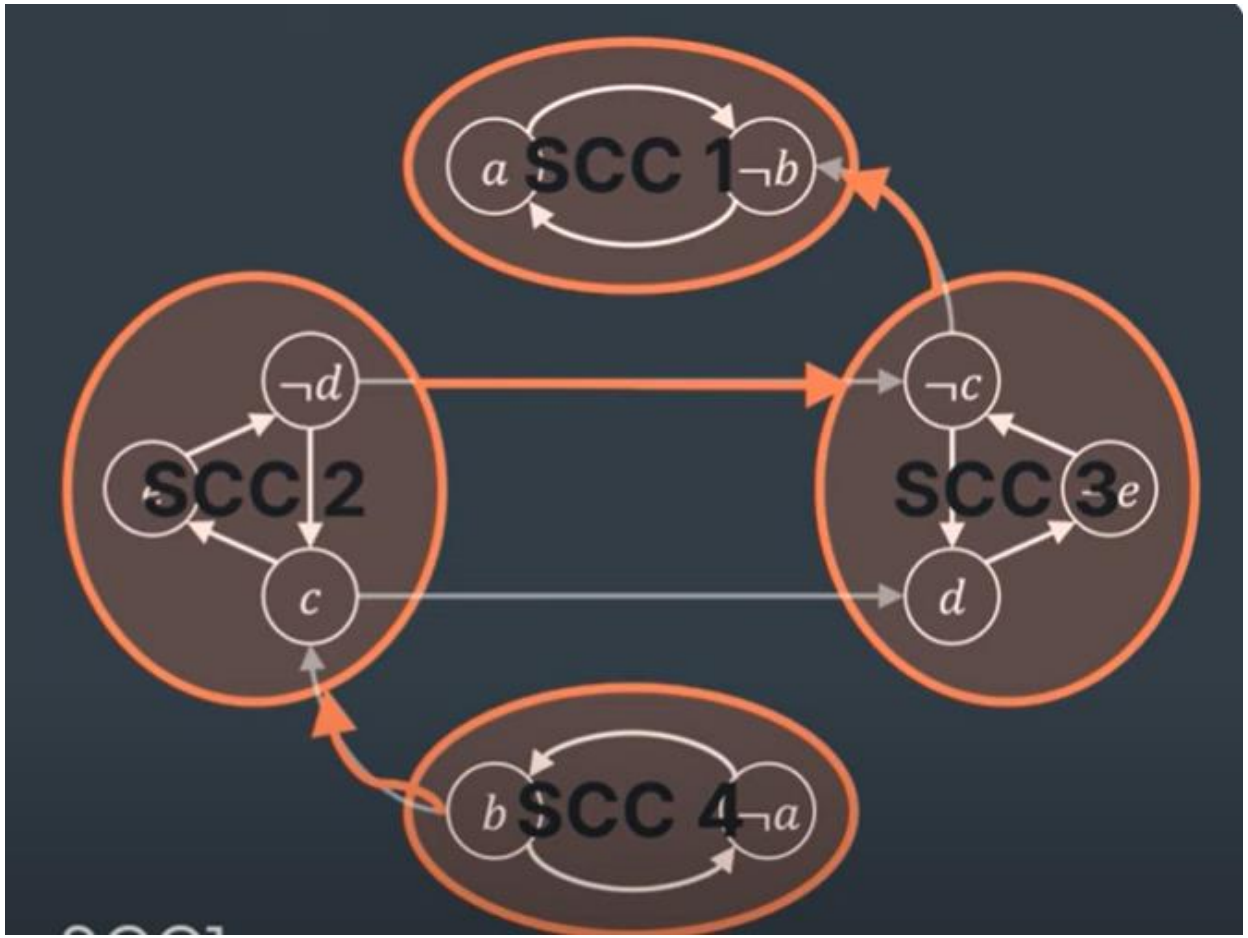
2-SAT

- $a \vee b$
- $\neg b \vee c$
- $\neg a \vee \neg b$
- $c \vee d$
- $\neg c \vee e$
- $\neg d \vee \neg e$
- $\neg c \vee d$



2-SAT

$a = T$
$b = F$
$c = F$
$d = T$
$e = F$



Topological order:
SCC4 – SCC2 – SCC3 – SCC1

2-SAT -> https://cses.fi/problemset/task/1684

```cpp
const ll N = 2e5+10;
char ans[N];
ll n,m,numcomp = 0;

vector<vector<int>> adj(N), adj_rev(N), adj_comp(N), SCC(N);
vector<bool> used(N);
vector<int> order, component, root(N);
ll in[N];

int flip(int x){
    return (x&1)?x+1:x-1;
}

void add_edge(char c1, int a, char c2, int b){
    a = 2*a - (c1 == '-');
    b = 2*b - (c2 == '-');
    adj[flip(a)].pb(b);
    adj[flip(b)].pb(a);
    adj_rev[b].pb(flip(a));
    adj_rev[a].pb(flip(b));
}

void dfs1(int v) {
    used[v] = true;

    for (auto u : adj[v])
        if (!used[u])
            dfs1(u);

    order.pb(v);
}

void dfs2(int v) {
    used[v] = true;
    component.pb(v);

    for (auto u : adj_rev[v])
        if (!used[u])
            dfs2(u);
}
```

```cpp
int main(){
    fast_cin();
    cin>>m>>n;
    for(int i = 0; i < m; i++){
        char c1,c2; ll a,b;
        cin>>c1>>a>>c2>>b;
        add_edge(c1,a,c2,b);
    }
    used.assign(2*n+5, false);
    for (int i = 1; i <= 2*n; i++){
        if (!used[i]){
            dfs1(i);
        }
    }
    used.assign(2*n+5, false);
    reverse(order.begin(), order.end());
    for (auto v : order){
        if (!used[v]){
            dfs2(v);
            numcomp++;
            for(auto x: component){
                root[x] = numcomp;
                adj_comp[numcomp].pb(x);
            }
            component.clear();
        }
    }
    for(int i = 1; i <= n; i++){
        if(root[2*i] == root[2*i-1]){
            cout<<"IMPOSSIBLE"<<endl;
            return 0;
        }
    }

    for(int i = 1; i <= 2*n; i++){
        for(auto x: adj[i]){
            if(root[i] != root[x]){
                SCC[root[i]].pb(root[x]);
                in[root[x]]++;
            }
        }
    }
```

```cpp
queue<int> cola;
order.clear();
for(int i = 1; i <= numcomp; i++){
    if(in[i] == 0){
        order.pb(i);
        cola.push(i);
    }
}

while(!cola.empty()){
    int curr = cola.front();
    cola.pop();
    for(auto x: SCC[curr]){
        in[x]--;
        if(in[x] == 0){
            cola.push(x);
            order.pb(x);
        }
    }
}

used.assign(2*n+5, false);
reverse(order.begin(), order.end());
for (auto v : order){
    for(auto x: adj_comp[v]){
        int i = (x+1)/2;
        if(!used[i]){
            ans[i] = (x&1 ?'-':'+');
            used[i] = true;
        }
    }
}
for(int i = 1; i <= n; i++){
    cout<<ans[i]<<" ";
}
return 0;
```